



نام دانشجو :

رضا ندیمی

شماره دانشجویی:

832204022

دانشکده سامانه های هوشمند - مکترونیک

تمرینات مکترونیک 2 سری ششم

تاریخ:

1405/03/28

استاد مربوطه:

دکتر تارویردی زاده

فضای توسعه ی مورد استفاده با توجه به عدم دسترسی به مخازن نرم افزار بین المللی.....	3
1. دو پالس PWM در دو پایه میکرو کنترلر به روش زیر ایجاد نمایید (فرکانس هر دو پالس بایستی 2 KHz باشد): - با تنظیم . و ۱ کردن 12 پایه <PB11:PBO>، اندازه Duty Cycle مربوط به پالس PWM را تنظیم نمایید. Duty Cycle مربوط به پایه دیگر باشد یعنی با افزایش Duty Cycle ایجاد شده در یک پایه، مکمل Duty Cycle - در یکی از پایه ها، Duty Cycle در پایه دیگر کاهش یابد و برعکس.....	4
بخش اول: اطلاعات کلی پروژه (General Information)	4
بخش دوم: تنظیمات انجام شده در (STM32CubeMX Configs) STM32CubeMX	4
بخش سوم: کد Keil و توضیحات آن (Keil Code & Explanation)	6
بخش چهارم: تنظیمات و شبیه سازی در پروتئوس (Proteus Setup and Configuration)	8
2. دو پالس PWM در دو پایه میکرو کنترلر به روش زیر ایجاد نمایید (فرکانس هر دو پالس بایستی 2 KHz باشد): - دو عدد پتانسیومتر در پایه های ADC1_IN1 و ADC1_IN2 قرار دهید با چرخاندن هر پتانسیومتر، ولتاژ آنالوگ مربوطه، در این پایه های آنالوگ را با استفاده از ماژول ADC میکرو کنترلر بخوانید و متناظر با اندازه ولتاژ این پایه های آنالوگ، Duty Cycle سیگنال PWM تولید شده در پایه های متناظر را تنظیم نمایید. - هر پتانسیومتر متصل به پایه های ADC1_IN1 و ADC1_IN2 را برای تنظیم Duty Cycle یکی از پایه ها، مورد استفاده قرار دهید. دقت داشته باشید که با توجه دقت تنظیم Duty Cycle را حداقل ۱۲ بیتی در نظر داشته باشید.	11
(1 اطلاعات کلی پروژه (General Information)	11
(2 تنظیمات انجام شده در (STM32CubeMX Configs) STM32CubeMX	11
(3 کد Keil 13	
(4 تنظیمات و شبیه سازی در پروتئوس (Proteus Setup)	15
3. اعدادی بصورت نمودار مقابل (با زمانبندی مناسب) در نرم افزار Matlab، در کامپیوتر ایجاد نمایید، این اعداد را در همان حین تولید به میکرو کنترلر بفرستید و Duty Cycle موج PWM را به این روش براساس مقادیر اعداد ارسالی از نرم افزار Matlab، تغییر دهید. هدف از این تمرین: ارسال اعداد بیشتر از ۸ بیت از طریق پروتکل ۸ بیتی ارسال داده (UART) میباشد. دقت: این تمرین را از طریق ارسال داده در قالب ۲ یا چند بایت، حل نمایید....	16
بخش اول: تنظیمات STM32CubeMX	16
بخش دوم: برنامه نویسی در نرم افزار Keil	19
4. دو پالس PWM در دو پایه میکرو کنترلر به روش زیر ایجاد نمایید (فرکانس هر دو پالس بایستی 2 KHz باشد): - اعدادی بصورت ۲ نمودار زیر (با زمانبندی مناسب) در نرم افزار Matlab، در کامپیوتر ایجاد نمایید، این اعداد را در همان حین تولید به میکرو کنترلر بفرستید و Duty Cycle موج PWM را به این روش تغییر دهید. - هر يك از ۲ نمودار زیر را برای تنظیم Duty Cycle، یکی از موج های PWM تولید شده در پایه ها استفاده نمایید.....	28
بخش اول تنظیمات میکرو کنترلر در CubeMX	28
بخش دوم کدهای Keil	30
بخش سوم اسکریپت متلب	34
بخش چهارم شبیه سازی مدار	36

5. از طریق تایمر ۲ یعنی (TIM2)، یک عدد PWM روی پایه PA15 تولید کنید (یک سیگنال PWM با فرکانس 50Hz و Duty Cycle برابر با ۵۰٪). این پایه را با یک مقاومت ۱۵۰ اهمی، به پایه PA8 متصل کنید (برای جلوگیری از اتصال کوتاه احتمالی در اتصال ۲ تا پایه از میکرو کنترلر به همدیگر). در پایه PA8 (از طریق Capture در TIM1) به اندازه گیری فرکانس و عرض پالس (Duty Cycle) تولید شده در پایه PA15، پردازید و آنها را روی LCD نمایش دهید (سطر اول LCD، مقدار فرکانس و سطر دوم LCD، عرض پالس را برحسب میلی ثانیه نمایش دهید)..... 37
6. تمرین فوق را برای یک سیگنال PWM با فرکانس 5kHz و Duty Cycle برابر با ۲۰٪، انجام دهید (فرکانس و عرض پالس (Duty Cycle) تولید شده را اندازه گیری کرده و آنها را روی LCD نمایش دهید)..... 44
۱. کلیات پروژه 44
۲. تنظیمات نرم افزار STM32CubeMX 44
۳. برنامه نویسی در نرم افزار Keil 46
۴. شبیه سازی در نرم افزار Proteus 50
7. تمرین فوق را برای یک سیگنال PWM با فرکانس 50kHz و Duty Cycle برابر با ۵٪ انجام دهید (فرکانس و عرض پالس (Duty Cycle) تولید شده را اندازه گیری کرده و آنها را روی LCD نمایش دهید). مهم: در خصوص افزایش فرکانس پالس و کاهش عرض آن، و امکان اندازه گیری آنها، بحث و تشریح کنید..... 51
1. اطلاعات عمومی و محاسبات طراحی 51
2. تنظیمات STM32CubeMX 54
3. برنامه نویسی در Keil 56
4. شبیه سازی در Proteus 60
8. DMA را فعال کرده و سوال شماره ۶ را به این روش حل کنید. مهم: در خصوص افزایش فرکانس پالس و کاهش عرض آن، و امکان اندازه گیری آنها با فعال بودن DMA، بحث و تشریح کنید..... 63
- بخش اول: اطلاعات کلی و محاسبات ریاضی 63
- بخش سوم: کد keil 66
- بخش چهارم: شبیه سازی در پروتئوس 68

فضای توسعه‌ی مورد استفاده با توجه به عدم دسترسی به مخازن نرم افزار بین المللی

1. برای STM32 CubeMX پک F1 نسخه ی V1.8.0

2. برای Keil پک Keil.STM32F1xx_DFP.2.4.0

3. برای Keil پک ARM.CMSIS.4.5.0 متعلق به سال 2015 (آخرین نسخه ای که در نت ملی توانستم

پیدا کنم. الان سال 2026 می باشد یعنی چیزی حدود 11 سال پیش)

4. برای ارتباط سریال با میکرو کنترلر از virtual serial driver pro استفاده شده است که از لینک زیر
دانلود شده است.

<https://p30download.ir/fa/entry/65323/virtual-serial-port-driver>

1. دو پالس PWM در دو پایه میکرو کنترلر به روش زیر ایجاد نمایید (فرکانس هر دو پالس بایستی 2 KHz باشد): - با تنظیم ۱ و ۱۲ کردن پایه <PB11:PBO>، اندازه Duty Cycle مربوط به پالس PWM را تنظیم نمایید. Duty Cycle مربوط به پایه دیگر باشد یعنی با افزایش Duty Cycle ایجاد شده در یک پایه، مکمل Duty Cycle - در یکی از پایه ها، Duty Cycle در پایه دیگر کاهش یابد و برعکس.

بخش اول: اطلاعات کلی پروژه (General Information)

هدف از این پروژه، تولید دو سیگنال PWM کاملاً مکمل (Complementary) با فرکانس دقیق ۲ کیلوهرتز (2 KHz) و با رزولوشن حداقل ۱۲ بیت (12-bit) با استفاده از میکرو کنترلر STM32F103C6 است. در این سیستم، چرخه کار (Duty Cycle) موج های خروجی توسط یک عدد باینری ۱۲ بیتی کنترل می شود. این ۱۲ بیت از طریق پایه های PBO تا PB11 به عنوان ورودی خوانده می شوند. خروجی های PWM نیز روی پایه های PA0 و PA1 تولید می گردند. برای ایجاد دو موج کاملاً قرینه (مکمل) و جلوگیری از باگ های شبیه ساز پروتئوس، به جای تغییر حالت PWM، از تکنیک سخت افزاری «معکوس کردن قطبیت خروجی (Polarity Inversion)» استفاده شده است.

بخش دوم: تنظیمات انجام شده در STM32CubeMX (STM32CubeMX Configs)

برای رسیدن به اهداف پروژه، پیکربندی های زیر در نرم افزار CubeMX انجام شده است:

1. تنظیمات کلاک: (Clock Configuration)

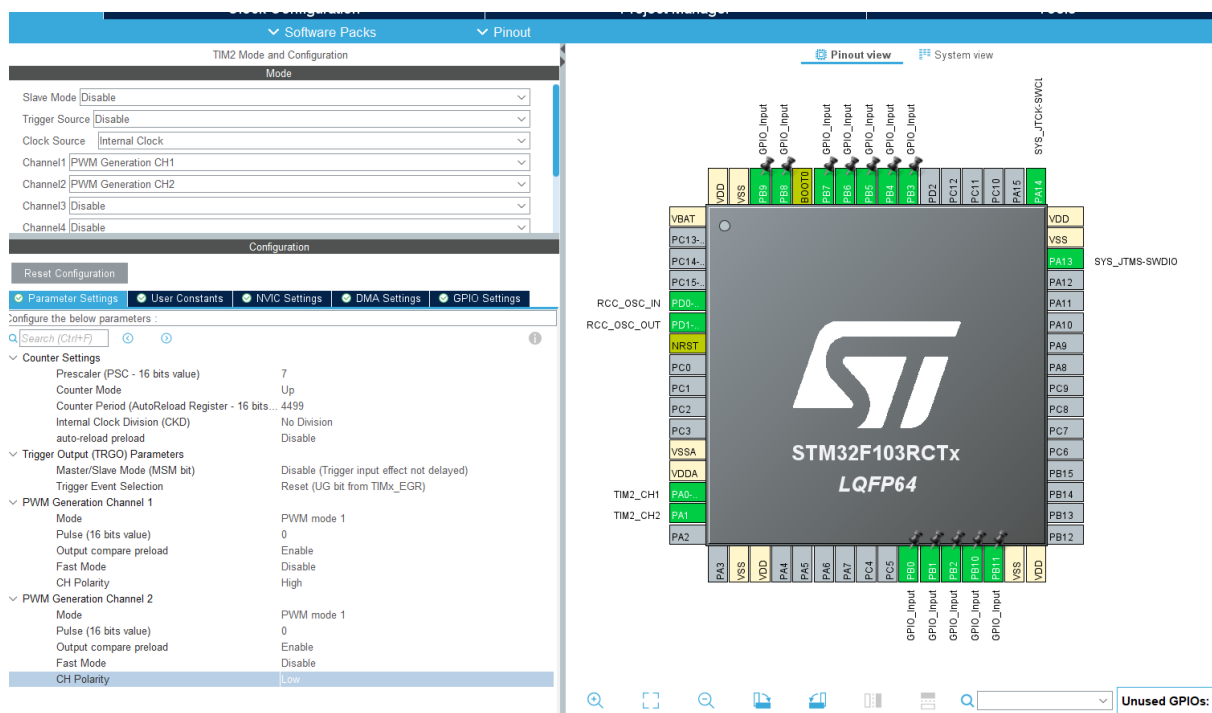
- منبع کلاک خارجی (HSE) روی کریستال 8 MHz تنظیم شده است.
- با استفاده از واحد PLL و ضریب ۹ (X9)، کلاک اصلی سیستم (SYSCLK) و تایمرها به 72 MHz ارتقا یافته است.

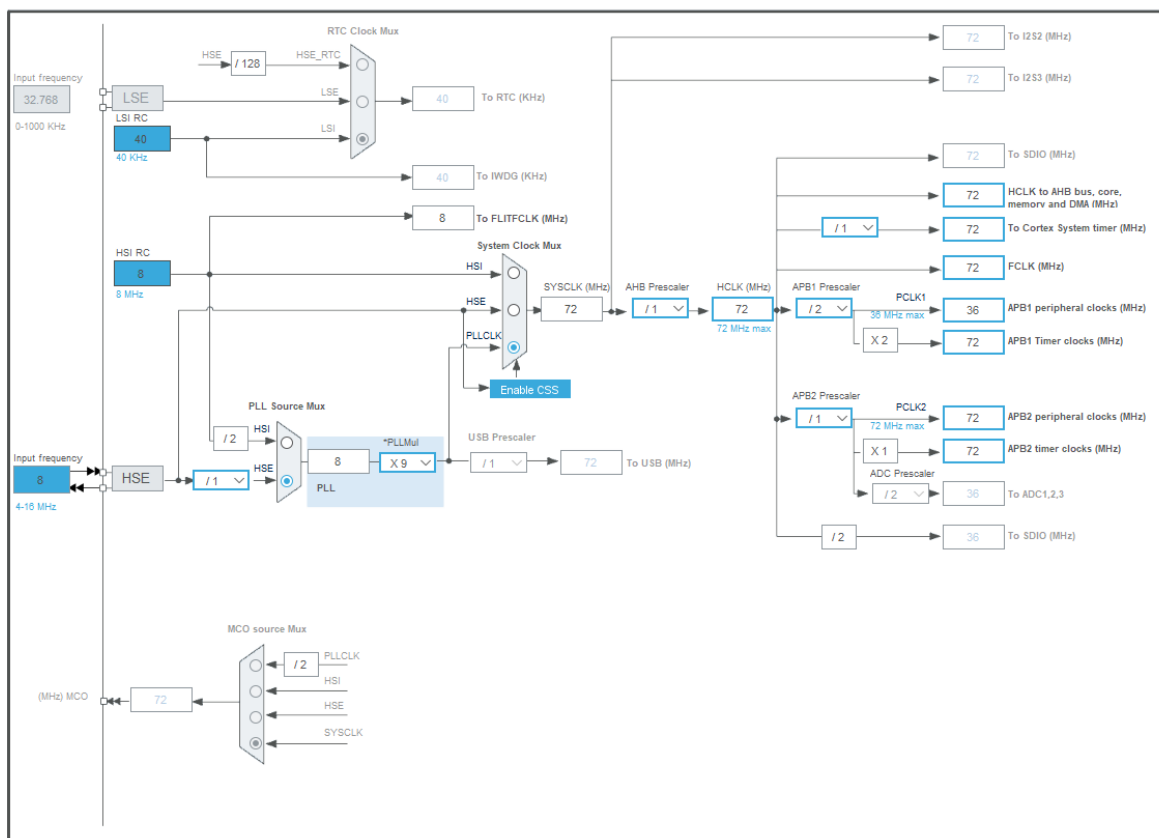
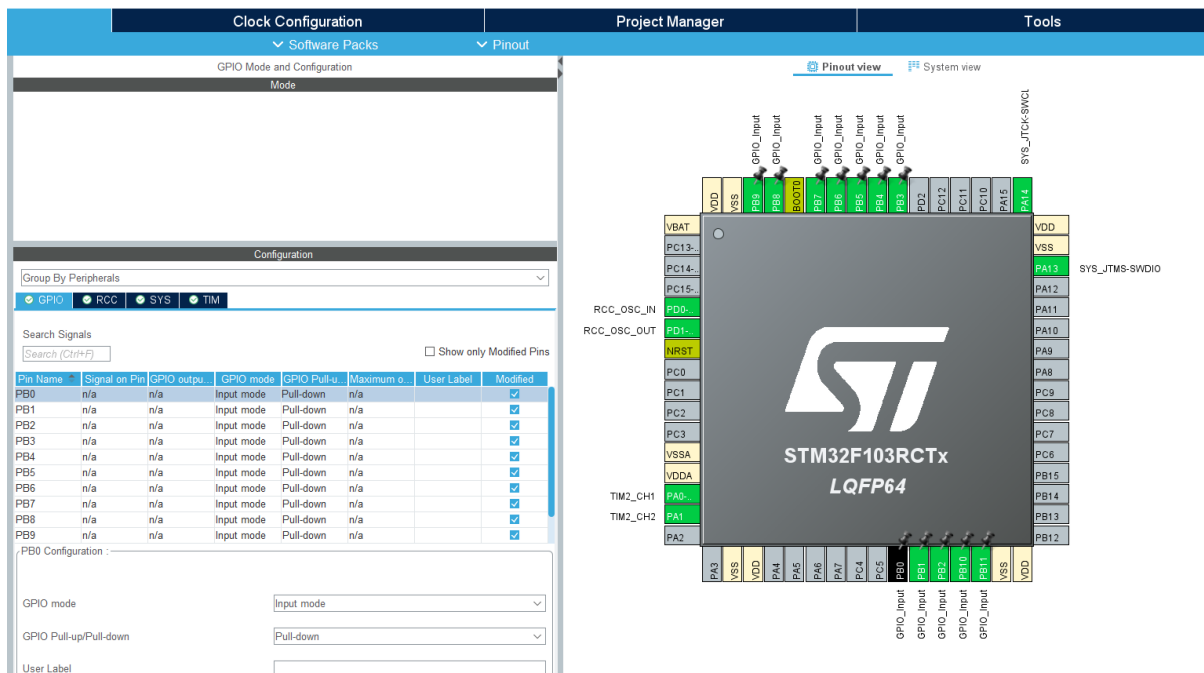
2. تنظیمات ورودی ها: (GPIO Configuration)

- پایه های PBO الی PB11 به عنوان ورودی (Input) تعریف شده اند.
- مقاومت های داخلی به حالت Pull-down تنظیم شده اند تا زمانی که کلیدها باز هستند، وضعیت پایه به صورت پیش فرض صفر منطقی (LOW) باشد و از نویز جلوگیری شود.

3. تنظیمات تایمر ۲: (TIM2 Configuration)

- منبع کلاک تایمر روی **Internal Clock** تنظیم شده است.
- کانال ۱ و کانال ۲ روی حالت **PWM Generation** تنظیم شده‌اند.
- **Prescaler** (مقسم فرکانس): برابر با ۷ تنظیم شده است.
- **Counter Period (ARR)**: برابر با ۴۴۹۹ تنظیم شده است. (فرمول فرکانس: فرکانس کلاک سیستم ۷۲ مگاهرتز تقسیم بر (۴۵۰۰×۱) برابر با ۲۰۰۰ هرتز یا همان ۲ کیلوهرتز می‌شود).
- **PWM Mode**: برای هر دو کانال روی **PWM mode 1** قرار دارد.
- **CH Polarity** (مهم‌ترین بخش برای موج مکمل): قطبیت کانال ۱ روی **High** و قطبیت کانال ۲ روی **Low** تنظیم شده است. این کار باعث می‌شود سخت‌افزار به صورت خودکار سیگنال کانال ۲ را معکوس کند.





بخش سوم: کد Keil و توضیحات آن (Keil Code & Explanation)

در محیط نرم افزار Keil، منطق اصلی برنامه درون حلقه بی نهایت (1) while پیاده سازی شده است. از آنجایی که معکوس سازی (Inversion) سیگنال ها به سخت افزار سپرده شده است، نیازی به محاسبات تفریقی در نرم افزار نداریم و صرفاً مقدار محاسبه شده را به هر دو کانال ارسال می کنیم.

کد زبان C برای قرار دادن در حلقه اصلی:

```
/* USER CODE BEGIN 2 */
// Start the PWM signals for Channel 1 and Channel 2
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    // 1. Read the entire Port B register and mask the first 12 bits (PB0 to PB11)
    // 0xFFFF in hex equals 0000 1111 1111 1111 in binary
    uint16_t portB_read = GPIOB->IDR & 0xFFFF;

    // 2. Map the 12-bit input (0 to 4095) to our Timer ARR range (0 to 4500)
    // Casting to uint32_t is important to prevent arithmetic overflow during
    multiplication
    uint32_t mapped_duty = ((uint32_t)portB_read * 4500) / 4095;

    // 3. Set the calculated duty cycle for Channel 1
    // The hardware will output this normally (Active HIGH)
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, mapped_duty);

    // 4. Set the exact same duty cycle for Channel 2
    // Because we set Channel 2 Polarity to LOW in CubeMX, the hardware
    automatically inverts it
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, mapped_duty);
```

```
// Small delay added to keep the Proteus simulation running smoothly and
prevent CPU overload
HAL_Delay(10);
}
/* USER CODE END 3 */
```

توضیح منطق کد:

1. در ابتدا کل رجیستر ورودی پورت B (GPIOB->IDR) خوانده می‌شود. با استفاده از عملگر & و ماسک 0x0FFF، تنها وضعیت ۱۲ پایه اول (مقدار صفر تا ۴۰۹۵) استخراج می‌شود و بیت‌های اضافی نادیده گرفته می‌شوند.
2. چون بازه ورودی ما بین ۰ تا ۴۰۹۵ است، اما سقف تایمر ما (ARR) عدد ۴۵۰۰ است، با استفاده از یک تناسب ساده ریاضی (ضرب در ۴۵۰۰ و تقسیم بر ۴۰۹۵) مقدار ورودی را به بازه دیوتی ساینکل تایمر نگاشت (Map) می‌کنیم. استفاده از متغیر ۳۲ بیتی از سرریز شدن عدد در هنگام ضرب جلوگیری می‌کند.
3. مقدار محاسبه شده (mapped_duty) توسط تابع HAL_TIM_SET_COMPARE__ به هر دو کانال تایمر داده می‌شود تا پالس‌ها ایجاد شوند.

بخش چهارم: تنظیمات و شبیه‌سازی در پروتئوس (Proteus Setup and Configuration)

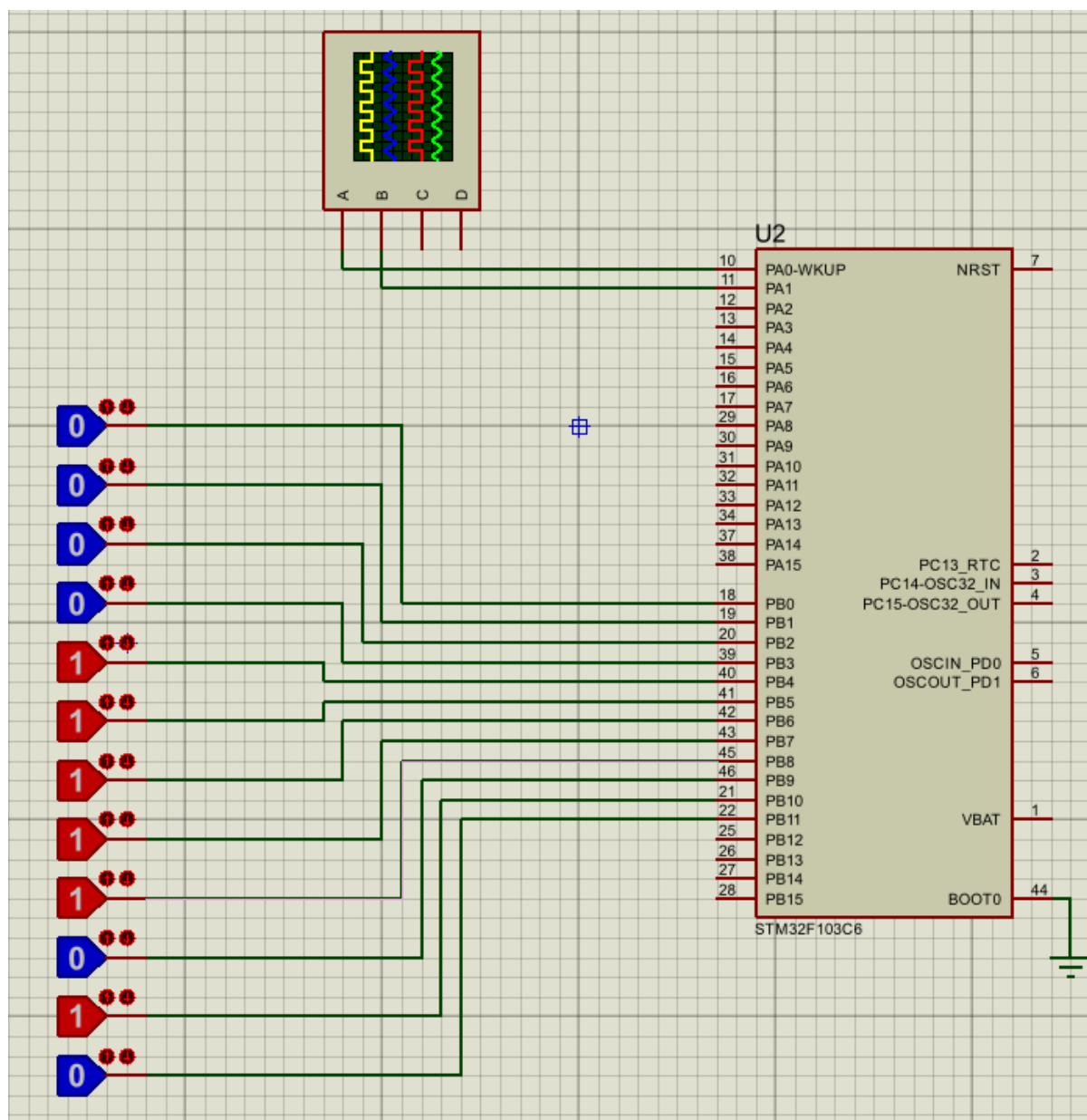
برای تست و مشاهده عملکرد سیستم، محیط پروتئوس به شکل زیر پیکربندی شده است:

1. طراحی مدار * :قطعه STM32F103C6 به صفحه کار اضافه شده است.
 - تعداد ۱۲ عدد Logic State (برای شبیه‌سازی ورودی‌های صفر و یک دیجیتال) به پایه‌های PB0 تا PB11 متصل شده‌اند.
 - یک دستگاه اسیلوسکوپ (Oscilloscope) به مدار اضافه شده است. پایه PA0 (خروجی کانال ۱) به کانال A اسیلوسکوپ و پایه PA1 (خروجی کانال ۲) به کانال B اسیلوسکوپ متصل گردیده است.

2. پیکربندی میکروکنترلر در پروتئوس:

- با دوبار کلیک بر روی میکروکنترلر STM32، پنجره تنظیمات آن باز می‌شود.

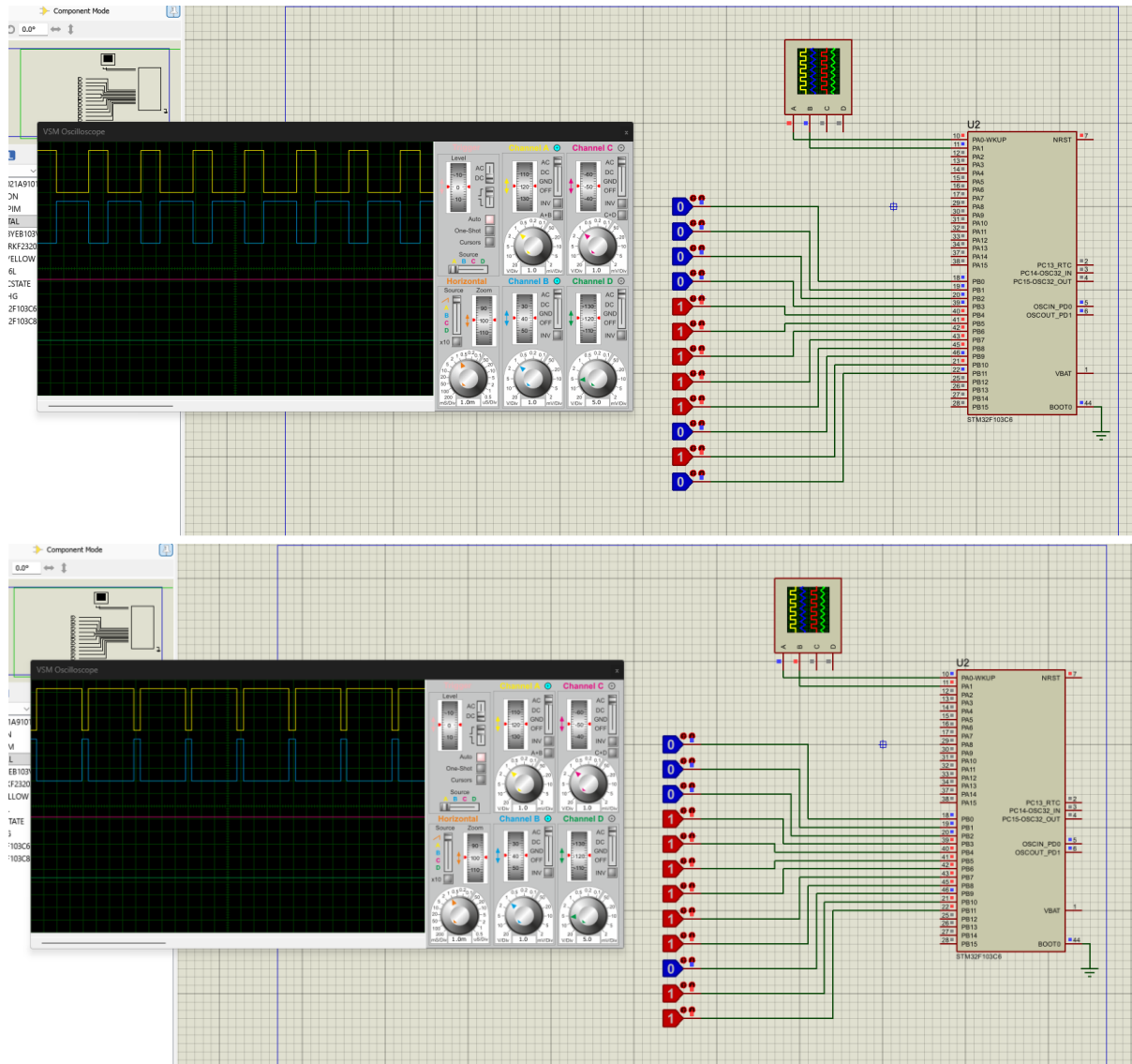
- در بخش **Program File**، مسیر پوشه پروژه معرفی شده و فایل با پسوند **.hex** (که در پوشه **Objects** توسط Keil کامپایل شده) انتخاب می‌گردد.
- در بخش **Crystal Frequency** (در صورت نیاز)، مقدار کریستال روی **8MHz** تنظیم می‌شود تا شبیه‌ساز با کلاک تعریف شده در CubeMX هماهنگ باشد.



3. اجرای شبیه‌سازی:

- با اجرای شبیه‌سازی (Play)، اسیلوسکوپ دو موج مربعی زرد و آبی رنگ را نشان می‌دهد.
- به دلیل تنظیم قطبیت معکوس در سخت‌افزار، این دو موج کاملاً خلاف یکدیگر (مکمل) هستند؛ یعنی هر زمان که کانال A بالا (High) است، کانال B پایین (Low) است.

- با تغییر دادن مقادیر 0 و 1 در ابزارهای Logic State ، عرض پالس‌ها (Duty Cycle) به صورت زنده و با دقت ۱۲ بیتی تغییر می‌کند در حالی که فرکانس ثابت روی ۲ کیلوهرتز باقی می‌ماند.



ویدیو ی شبیه سازی در پوشه سوال مربوطه موجود می باشد

2. دو پالس PWM در دو پایه میکروکنترلر به روش زیر ایجاد نمایید (فرکانس هر دو پالس بایستی 2 KHz باشد): - دو عدد پتانسیومتر در پایه های ADC1_IN1 و ADC1_IN2 قرار دهید با چرخاندن هر پتانسیومتر، ولتاژ آنالوگ مربوطه، در این پایه های آنالوگ را با استفاده از ماژول ADC میکروکنترلر بخوانید و متناظر با اندازه ولتاژ این پایه های آنالوگ، Duty Cycle سیگنال PWM تولید شده در پایه های متناظر را تنظیم نمایید. - هر پتانسیومتر متصل به پایه های ADC_IN1 و ADC1_IN2 را برای تنظیم Duty Cycle یکی از پایه ها، مورد استفاده قرار دهید. دقت داشته باشید که باتوجه دقت تنظیم Duty Cycle را حداقل ۱۲ بیتی در نظر داشته باشید.

(1) اطلاعات کلی پروژه (General Information)

در این بخش از پروژه، هدف تولید دو سیگنال PWM مستقل با فرکانس ۲ کیلوهرتز است که "Duty Cycle" هر کدام توسط یک پتانسیومتر آنالوگ به صورت مجزا کنترل می شود. برای بهینه سازی عملکرد و کاهش بار کاری پردازنده (CPU)، از ماژول ADC در حالت **Continuous** و انتقال داده توسط **DMA** استفاده شده است. این روش باعث می شود که خواندن داده های آنالوگ بدون درگیری مستقیم CPU انجام شود و به محض آماده شدن داده، از طریق تابع **Callback**، عرض پالس های خروجی به روزرسانی گردد.

(2) تنظیمات انجام شده در STM32CubeMX (STM32CubeMX Configs)

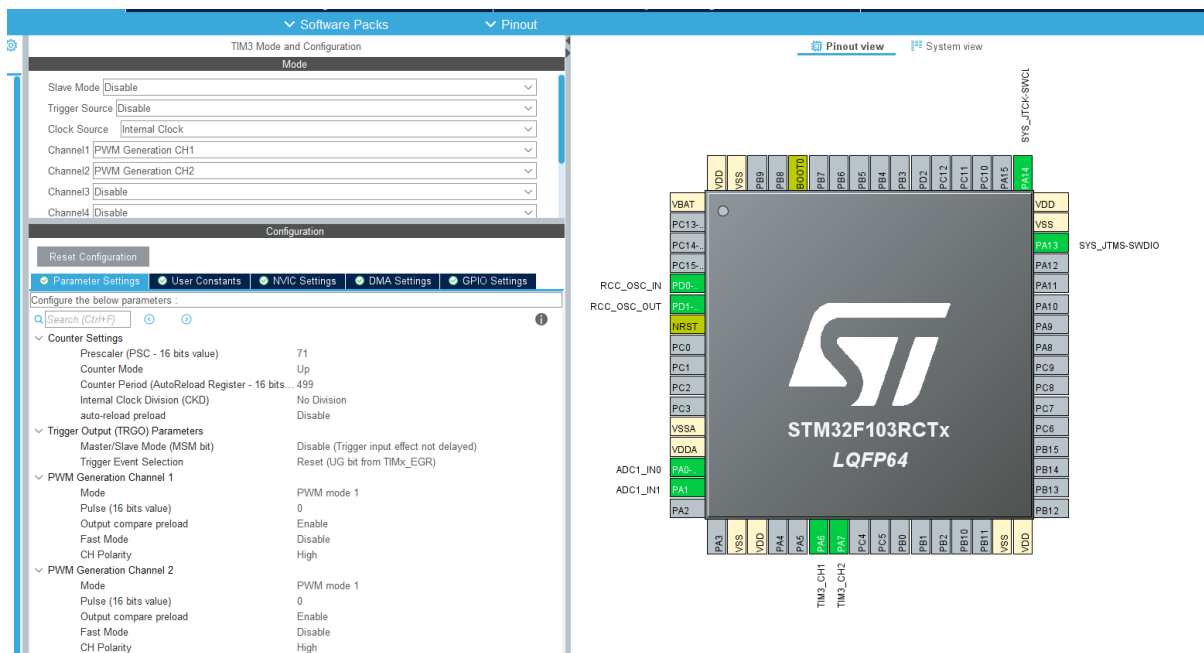
پیکربندی های زیر برای دستیابی به حداکثر پایداری در شبیه ساز پروتئوس انجام شده است:

• ADC1 Configuration : * فعال سازی IN0 و IN1 بر روی پایه های PA0 و PA1.

- Resolution: 12-bit.
- Enabled : Scan & Continuous Conversion Mode (برای خواندن متناوب هر دو کانال).
- DMA Settings: اضافه کردن ADC1 در حالت **Circular** (تا بافر به طور خودکار همیشه تازه بماند).

• TIM3 Configuration :

- فرکانس کلاک تایمر روی ۷۲ مگاهرتز تنظیم شده است.
- Prescaler: روی ۷۱ (تایمر را به ۱ مگاهرتز می رساند).
- Counter Period (ARR): در کد روی ۲۴۹ تنظیم می شود (تا فرکانس دقیق ۲ کیلوهرتز حاصل شود).
- پایه های خروجی PA6 و PA7 به عنوان کانال های PWM تعیین شده اند.



کد Keil (3)

در این پیاده‌سازی، از **DMA Callback** استفاده شده است. این کار باعث می‌شود به جای انتظار در حلقه **while** (Polling)، سخت‌افزار به محض تکمیل تبدیل ADC، پردازنده را مطلع کند.

```
/*USER CODE BEGIN PD*/

uint16_t adc_buffer[2]; /* [0] = PA0 (ADC_IN0), [1] = PA1 (ADC_IN1) */

/*USER CODE END PD*/

/*USER CODE BEGIN PV*/

ADC_HandleTypeDef hadc1;
DMA_HandleTypeDef hdma_adc1;
TIM_HandleTypeDef htim3;

/*USER CODE END PV*/

/*USER CODE BEGIN PFP*/

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc);

/*USER CODE END PFP*/
```

```

/* USER CODE BEGIN 0 */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if (hadc->Instance == ADC1)
    {
        uint32_t duty1 = ((uint32_t)adc_buffer[0] * 500) / 4096;
        uint32_t duty2 = ((uint32_t)adc_buffer[1] * 500) / 4096;

        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, duty1);
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, duty2);
    }
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buffer, 2);
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_2);
/* USER CODE END 2 */

```

توضیح منطق:

- تابع HAL_ADC_Start_DMA یک بار اجرا می شود و DMA کنترل انتقال داده از ADC به آرایه adc_buffer را بر عهده می گیرد.
- وقتی هر دو پتانسیومتر خوانده شدند، تابع HAL_ADC_ConvCpltCallback فراخوانی می شود.
- این تابع یک فراخوان بازگشتی است که توسط کتابخانه HAL و پس از تکمیل یک چرخه کامل تبدیل آنالوگ به دیجیتال به صورت خودکار صدا زده می شود
- زمان اجرا
- هرگاه تمام کانال های تعیین شده در حالت اسکن توسط ADC خوانده شوند و داده ها از طریق DMA به حافظه منتقل شوند این تابع بلافاصله اجرا می گردد
- در این برنامه این تابع مقدار خام هر دو پتانسیومتر را از بافر خوانده و متناسب با آن مقدار دیوتی سایکل دو کانال PWM را بدون دخالت پردازنده به روز رسانی می کند

(4) تنظیمات و شبیه‌سازی در پروتئوس (Proteus Setup)

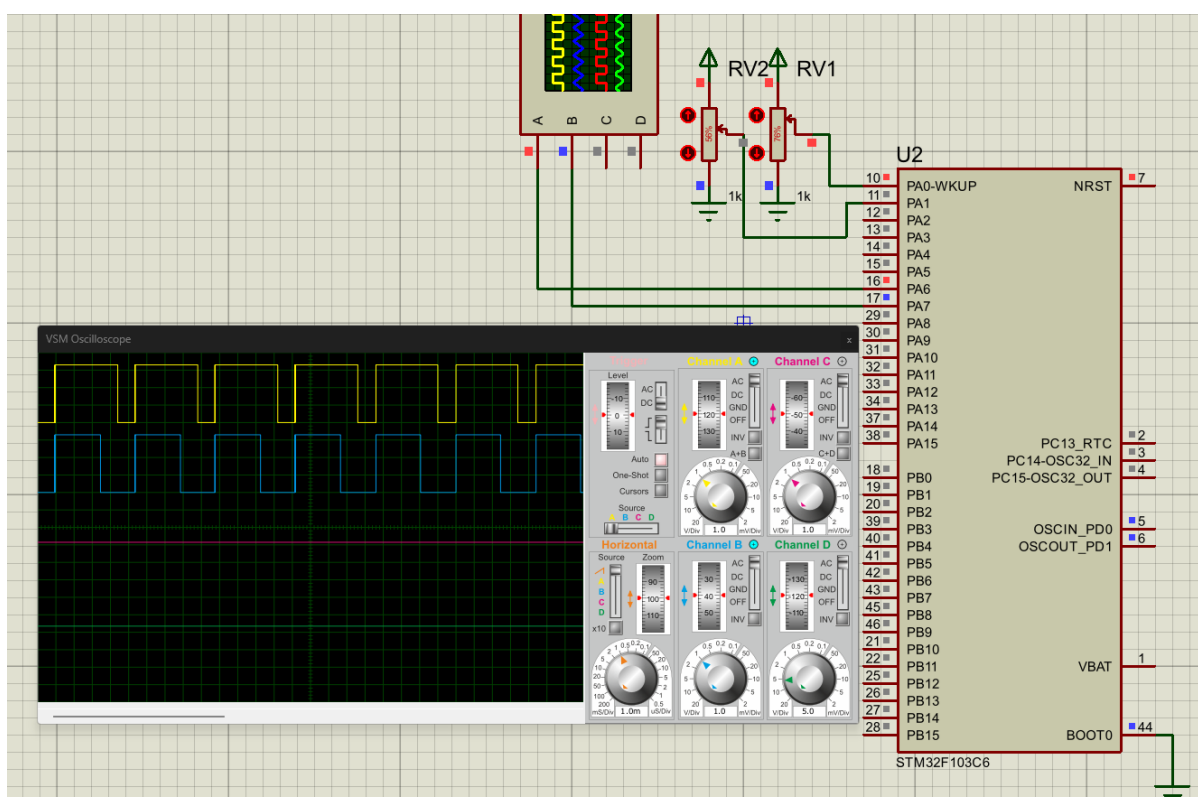
1. **اتصالات ***: دو پتانسیومتر را به PA0 و PA1 متصل کنید (پایه وسط به میکرو و دو سر دیگر به ۳.۳ ولت و زمین).

○ پایه‌های PA6 و PA7 را به کانال‌های A و B اسیلوسکوپ در پروتئوس وصل کنید.

2. **پیکربندی فایل Hex:** فایل hex تولید شده از Keil را در تنظیمات STM32 بارگذاری کنید.

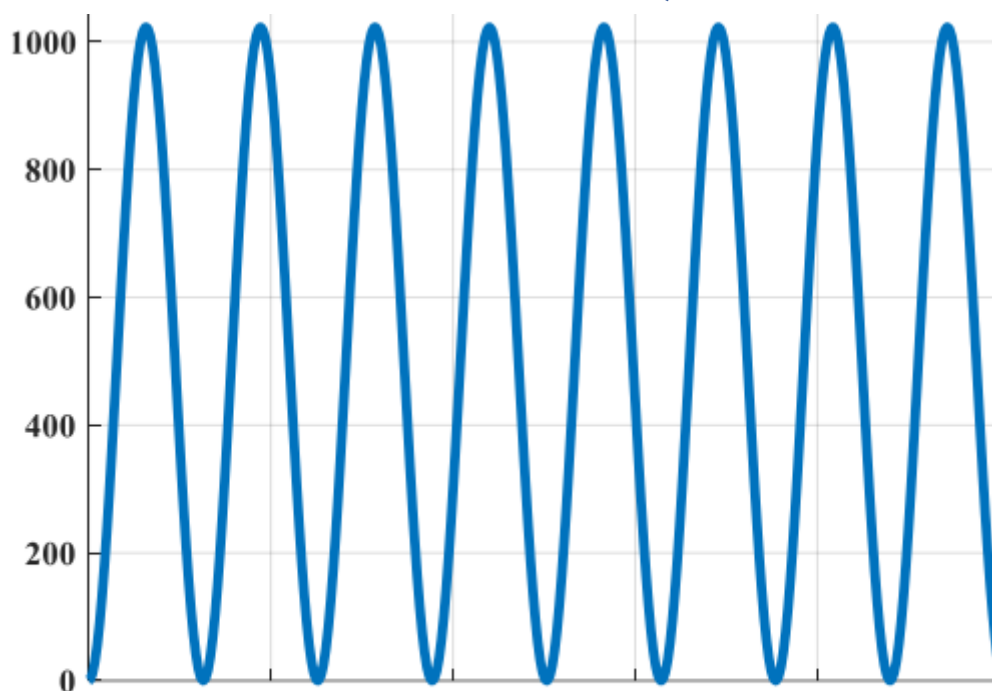
3. **اجرا:** با اجرای شبیه‌سازی، خواهید دید که تغییر پتانسیومترها بلافاصله عرض پالس‌ها را تغییر می‌دهد. به دلیل استفاده از DMA، پردازنده بسیار خنک‌تر عمل کرده و فشار کمتری به CPU سیستم شما در شبیه‌سازی وارد می‌شود.

تنظیمات Scale در اسیلوسکوپ پروتئوس روی مقادیر مناسب 1 ms/Div برای محور افقی قرار می‌دهیم



ویدیو ی شبیه سازی در پوشه سوال مربوطه موجود می باشد

3. اعدادی بصورت نمودار مقابل (با زمانبندی مناسب) در نرم افزار Matlab، در کامپیوتر ایجاد نمایید، این اعداد را در همان حین تولید به میکروکنترلر بفرستید و Duty Cycle موج PWM را به این روش براساس مقادیر اعداد ارسالی از نرم افزار Matlab، تغییر دهید. هدف از این تمرین: ارسال اعداد بیشتر از ۸ بیت از طریق پروتکل ۸ بیتی ارسال داده (UART) میباشد. دقت: این تمرین را از طریق ارسال داده در قالب ۲ یا چند بایت، حل نمایید.



نمودار فوق یک موج سینوسی، با حداکثر مقدار ۱۰۲۳ است (دقت ۱۰ بیتی در تنظیم Duty Cycle)

بخش اول: تنظیمات STM32CubeMX

در این بخش، میکروکنترلر **STM32F103C8** را برای دریافت داده‌ها از طریق UART و تولید موج PWM با دقت ۱۰ بیت (رزولوشن ۰ تا ۱۰۲۳) پیکربندی می‌کنیم.

۱. انتخاب میکروکنترلر و تنظیمات پایه (SYS و RCC):

- در بخش **SYS**، گزینه **Debug** را روی **Serial Wire** تنظیم کنید تا پایه‌های مربوطه رزرو شوند.
- در بخش **RCC**، منبع کلاک (HSE) را روی **Crystal/Ceramic Resonator** قرار دهید.

۲. پیکربندی تایمر (TIM2) برای تولید PWM:

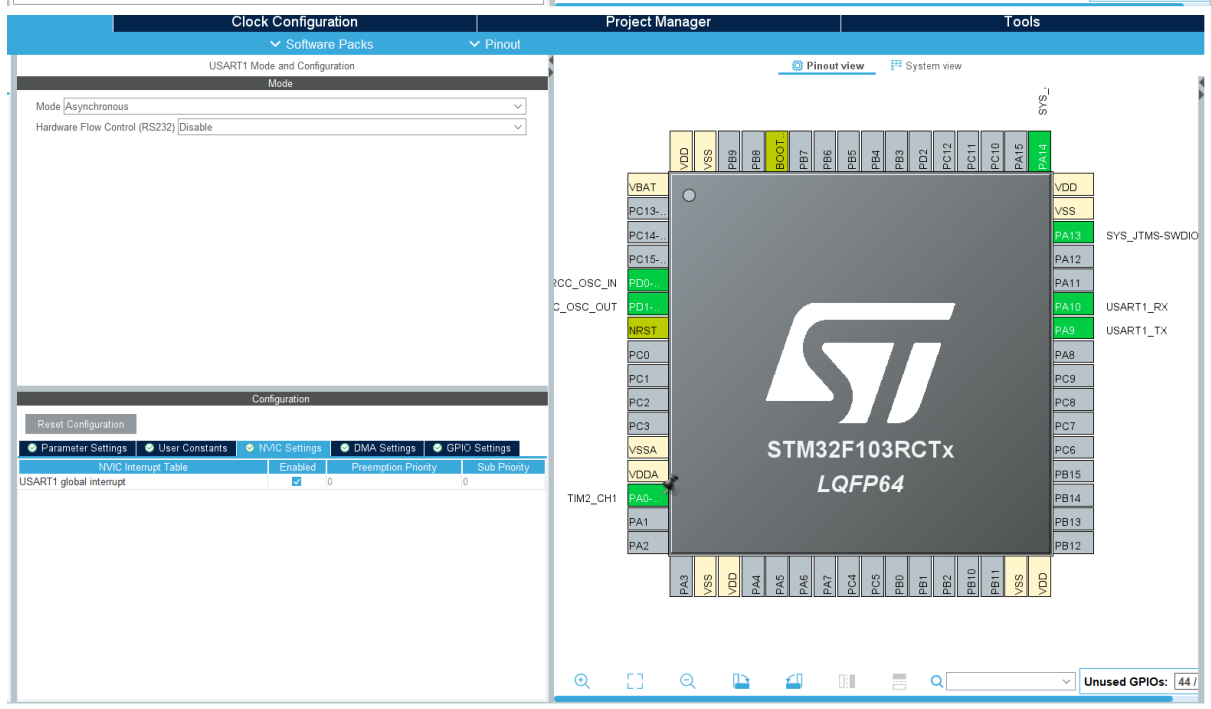
- در بخش **Timers**، گزینه **TIM2** را انتخاب کرده و **Clock Source** را روی **Internal Clock** تنظیم کنید.
- در کانال ۱ (Channel 1)، گزینه **PWM Generation CH1** را انتخاب کنید.
- **نکته در مورد پایه‌ها:** نرم‌افزار ممکن است به صورت خودکار پایه **PA15** را برای این کانال در نظر بگیرد. روی شکل میکروکنترلر کلیک چپ کرده و پایه **PA0** را به **TIM2_CH1** اختصاص دهید.
- تنظیمات پارامترها (**Parameter Settings**):
 - **Prescaler (PSC): 71** (برای کلاک ۷۲ مگاهرتز، فرکانس تایمر را به ۱ مگاهرتز کاهش می‌دهد).
 - **Counter Period (ARR): 1023** (برای ایجاد دقت ۱۰ بیتی).
 - **Pulse: 0** (برای شروع با **Duty Cycle** صفر).

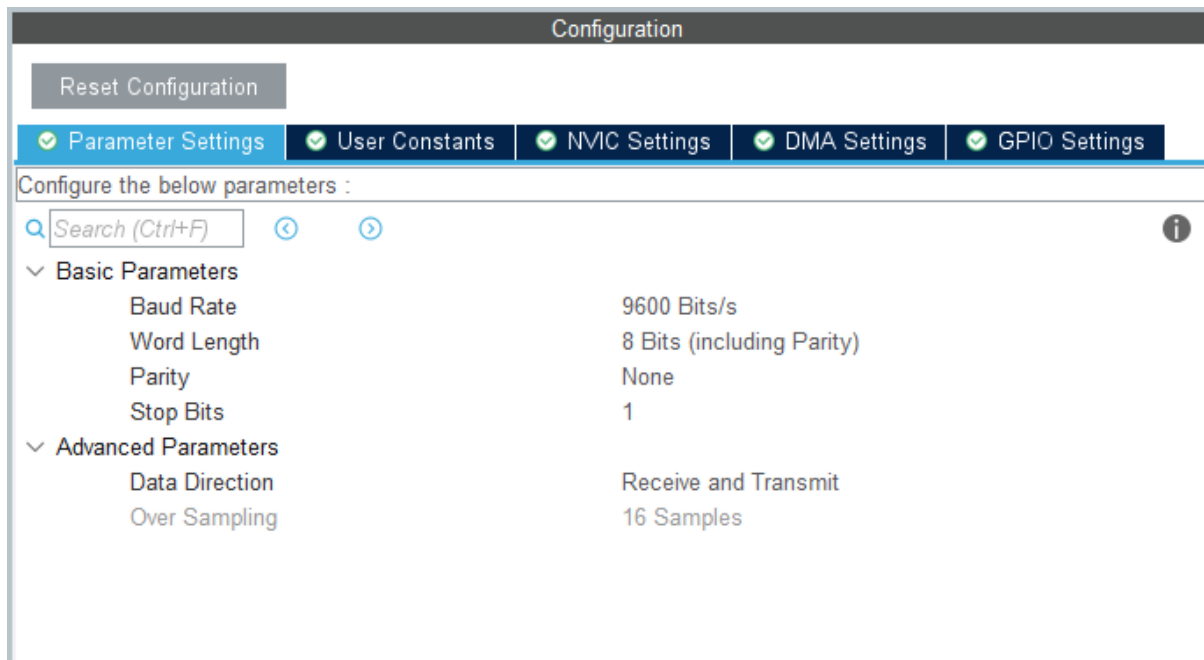
۳. پیکربندی ارتباط سریال (USART1):

- در بخش **Connectivity**، پورت **USART1** را در حالت **Asynchronous** تنظیم کنید.
- **Baud Rate** را روی 9600 قرار دهید.
- در تب **NVIC Settings**، حتماً تیک **USART1 global interrupt** را فعال کنید تا دریافت داده‌ها بدون توقف برنامه اصلی انجام شود.

۴. تنظیمات کلاک (Clock Configuration):

- در تب کلاک، در کادر **HCLK** عدد 72 را وارد کرده و **Enter** بزنید تا نرم‌افزار کلاک را روی ۷۲ مگاهرتز تنظیم کند.
- روی **Generate Code** کلیک کنید.





بخش دوم: برنامه‌نویسی در نرم‌افزار Keil

در این بخش، کدهای مربوط به دریافت داده‌ها را می‌نویسیم. برای جلوگیری از خطای همگام‌سازی، از تکنیک **کدگذاری ۷-بیتی** استفاده شده است. همچنین کدهایی برای رفع خطای **Overrun** (سرریز داده در شبیه‌ساز) گنجانده شده است.

فایل **main.c** را باز کرده و تغییرات زیر را اعمال کنید:

مرحله ۱: تعریف متغیرها

```
/* USER CODE BEGIN PV */
uint8_t rx_byte;    // Buffer to receive 1 byte at a time
uint16_t duty_cycle = 0; // The reconstructed 10-bit PWM value
/* USER CODE END PV */
```

مرحله ۲: راه‌اندازی اولیه تایمر و وقفه UART

```
/* USER CODE BEGIN 2 */
// Start the PWM on Timer 2 Channel 1 (PA0)
```

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
```

```
// Start listening for exactly 1 byte via Interrupt
```

```
HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
```

```
/* USER CODE END 2 */
```

مرحله ۳: توابع وقفه (Interrupt Callbacks)

کدهای زیر را در انتهای فایل قرار دهید. این بخش شامل سیستم امنیتی برای نادیده گرفتن خطای شبیه‌ساز و بازیابی خودکار ارتباط است.

```
/* USER CODE BEGIN 4 */
```

```
// This function is called every time 1 byte is successfully received
```

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
```

```
{
```

```
    if(huart->Instance == USART1)
```

```
    {
```

```
        // Force-clear the Overrun Error (ORE) flag so the simulation never gets stuck
```

```
        __HAL_UART_CLEAR_OREFLAG(huart);
```

```
        static uint8_t high_byte = 0;
```

```
        // Check if the highest bit is 1 (Marker bit for High Byte)
```

```
        if (rx_byte & 0x80)
```

```
        {
```

```
            // Remove the marker bit (0x80) and save as high_byte
```

```

        high_byte = rx_byte & 0x7F;
    }
    else
    {
        // It is the Low Byte (Highest bit is 0)
        // Reconstruct the 10-bit value
        duty_cycle = (high_byte << 7) | rx_byte;

        // Safety clamp to ensure it never exceeds 1023
        if (duty_cycle > 1023) {
            duty_cycle = 1023;
        }

        // Update the PWM Duty Cycle dynamically
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, duty_cycle);
    }

    // Listen for the next byte
    HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
}

void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART1)

```

```

{
    // Forcefully restart the UART interrupt listening process
    HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
}
}

/* USER CODE END 4 */

```

پس از قرار دادن کدها، دکمه **F7** را بزنید تا فایل hex کامپایل شود.

بخش سوم: اسکریپت MATLAB

وظیفه این اسکریپت، تولید موج سینوسی و ارسال آن بایت به بایت است. یک تأخیر ایمن pause(0.01) بین بایت‌ها قرار داده شده تا شبیه‌ساز Proteus فرصت پردازش داشته باشد و قفل نکند.

کدهای زیر را در یک اسکریپت جدید در متلب کپی کنید:

```

% --- SAFETY CLEANUP ---
% Ensures the COM port is released even if you close the window early
cleanupObj = onCleanup(@() closeSerialPort());

% --- SERIAL PORT SETUP ---
% Change "COM1" to the port MATLAB is using in your virtual pair (e.g. COM1)
s = serialport("COM1", 9600);

% --- SINE WAVE PARAMETERS ---
t = 0:0.05:100; % Time array (extended duration)
f = 0.2; % Frequency of the sine wave
% Generate wave: amplitude 511.5, offset by 511.5 to map strictly from 0 to 1023
wave_data = round(511.5 * sin(2*pi*f*t) + 511.5);

% --- UI SETUP ---
fig = figure('Name', '10-bit PWM Transmission');

```

```

keepRunning = true;

% Create a STOP button on the graph to gracefully exit the loop
btn = uicontrol('Style', 'pushbutton', 'String', 'STOP TRANSMISSION', ...
    'Position', [20 20 150 30], ...
    'Callback', 'keepRunning = false;');

hold on; title('Sending 10-bit Sine Wave to Proteus (7-Bit Encoded)');
xlabel('Time'); ylabel('Duty Cycle (0-1023)');

% --- TRANSMISSION LOOP ---
i = 1;
while keepRunning && i <= length(wave_data)

    val = wave_data(i);

    % --- THE 7-BIT ENCODING (Marker Bit System) ---
    % High Byte: Shift right 7 bits, then add 128 to force the MSB to 1
    high_byte = bitshift(val, -7) + 128;

    % Low Byte: Keep only the bottom 7 bits (0 to 127). MSB is naturally 0.
    low_byte = bitand(val, 127);

    % 1. Send High Byte first
    write(s, high_byte, "uint8");

    % 2. CRUCIAL SPACE: Give Proteus simulator time to process the first byte
    pause(0.01);

    % 3. Send Low Byte
    write(s, low_byte, "uint8");

    % Live plotting
    plot(t(1:i), wave_data(1:i), 'b-', 'LineWidth', 2);
    axis([0 max(t) 0 1100]);

```

```

drawnow;

% Delay to match physical real-time flow
pause(0.05);
i = i + 1;
end

disp('Transmission Complete or Stopped by User.');
```

```

% --- HELPER FUNCTION ---
function closeSerialPort()
    portList = serialportlist;
    if ~isempty(portList)
        clear s;
    end
    disp('COM Port closed safely and is ready for the next run.');
```

```

end
```

بخش چهارم: شبیه‌سازی در Proteus

مرحله نهایی، راه‌اندازی مدار در شبیه‌ساز و اتصال آن به متلب است.

۱. قطعات و سیم‌کشی:

- قطعه **STM32F103C8** را به همراه یک **COMPIM** و یک **Oscilloscope** به صفحه اضافه کنید.

- پایه **TXD** از قطعه **COMPIM** را به پایه **PA10** میکروکنترلر وصل کنید.

- پایه **PA0** میکروکنترلر را به کانال **A** در اسیلوسکوپ متصل کنید.

۲. تنظیمات میکروکنترلر (رفع مشکل Baud Rate):

- روی **STM32** دو بار کلیک کنید و در بخش **Program File**، فایل **hex** تولید شده توسط **Keil** را قرار دهید.

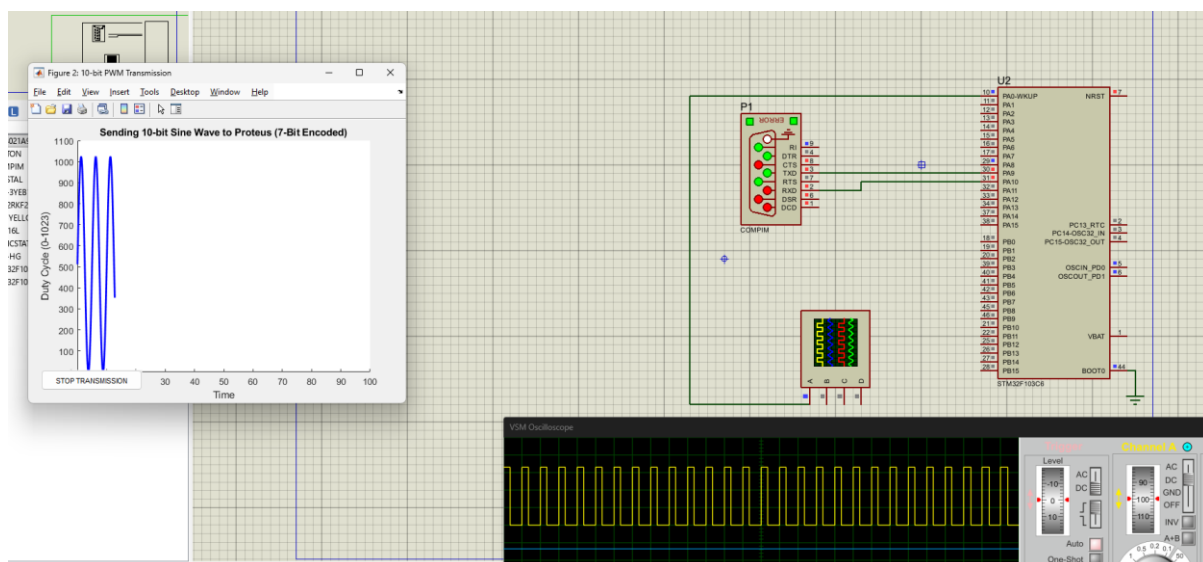
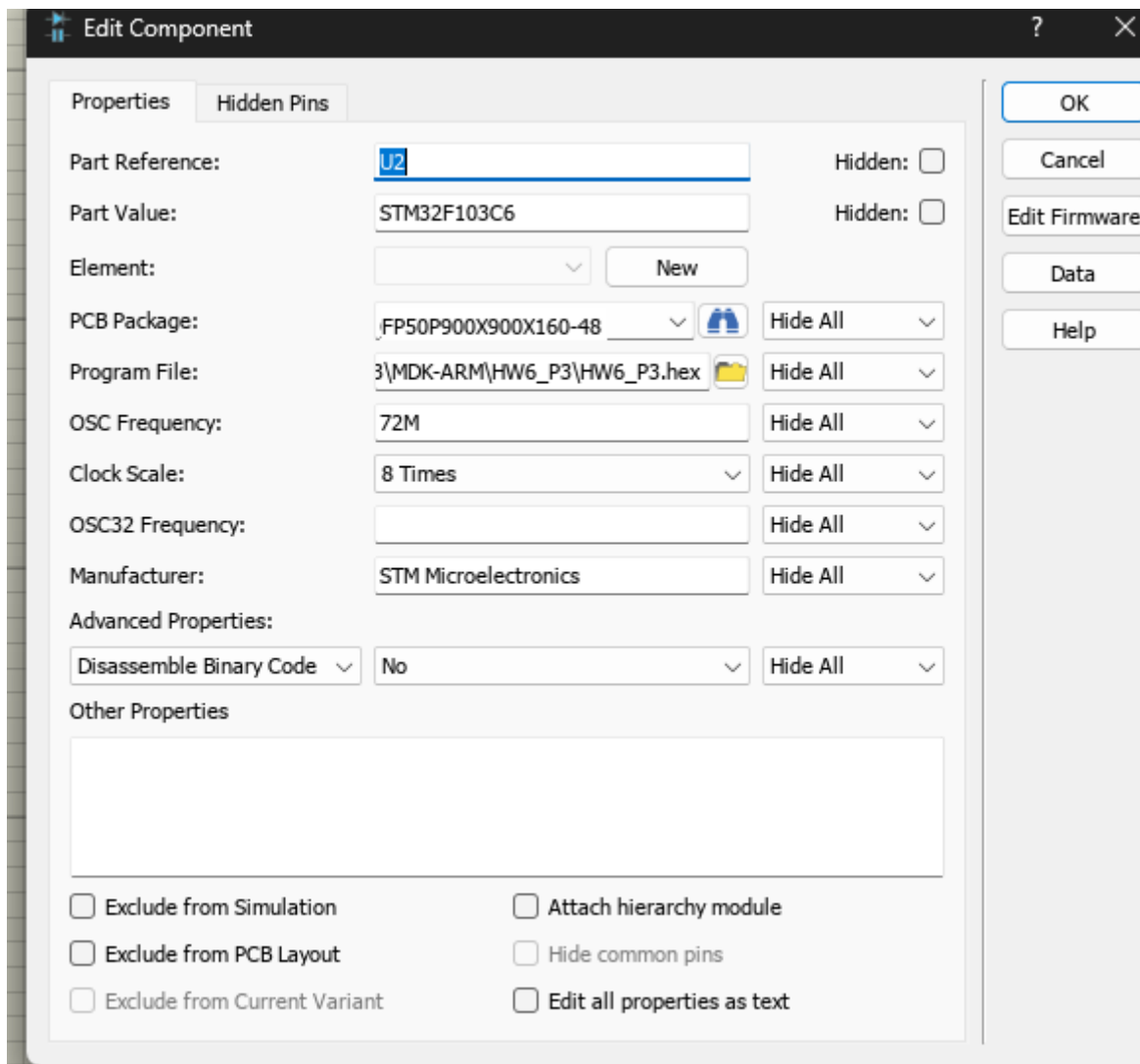
- بسیار مهم: مقدار Crystal Frequency را حتماً روی M72 تنظیم کنید. این کلاک باید دقیقاً با کلاکی که در نرم‌افزار CubeMX تنظیم کردید برابر باشد، در غیر این صورت ارتباط سریال قطع خواهد شد.

۳. تنظیمات COMPIIM:

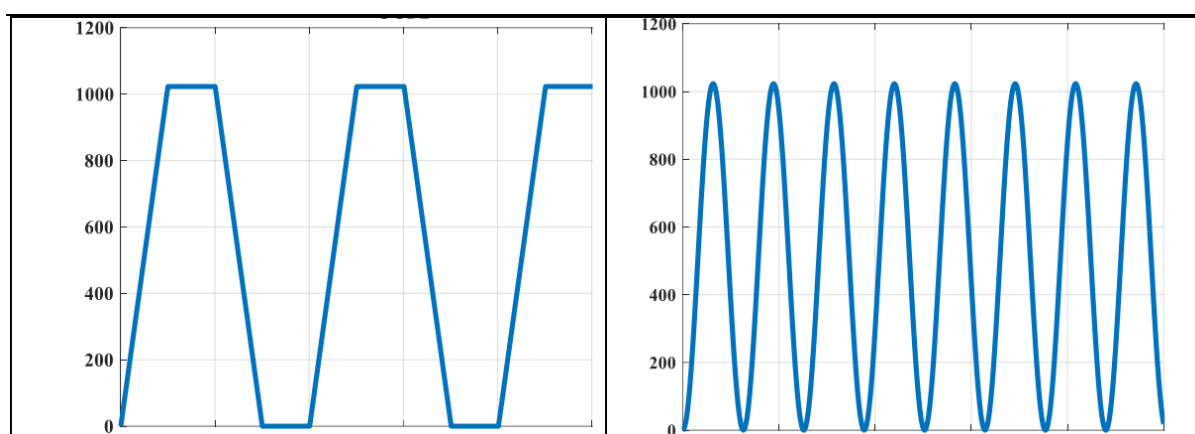
- در تنظیمات COMPIIM، پورت را روی جفت پورت متلب تنظیم کنید (مثلاً اگر متلب روی COM1 است، اینجا COM2 را انتخاب کنید).
- هر دو مقدار Physical Baud Rate و Virtual Baud Rate را روی 9600 قرار دهید.

۴. اجرای نهایی:

- ابتدا دکمه Play را در نرم‌افزار Proteus بزنید.
- سپس برنامه متلب را اجرا (Run) کنید.
- با مشاهده اسیلوسکوپ خواهید دید که تغییرات موج مربعی کاملاً همگام با موج سینوسی متلب رسم می‌شود و عرض پالس‌ها (Duty Cycle) به وضوح از یکدیگر تفکیک شده‌اند.



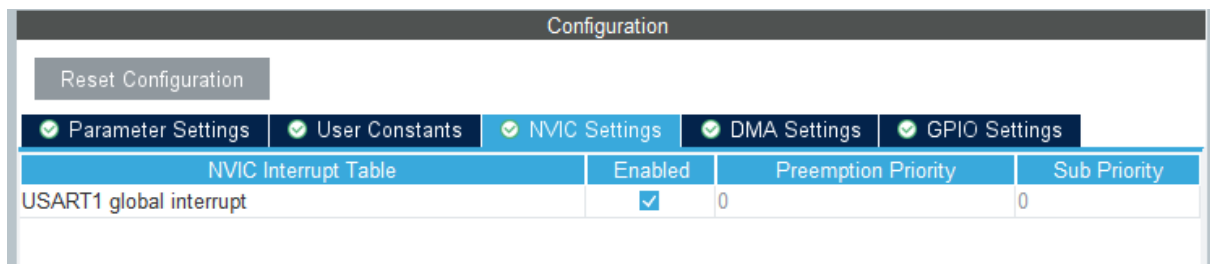
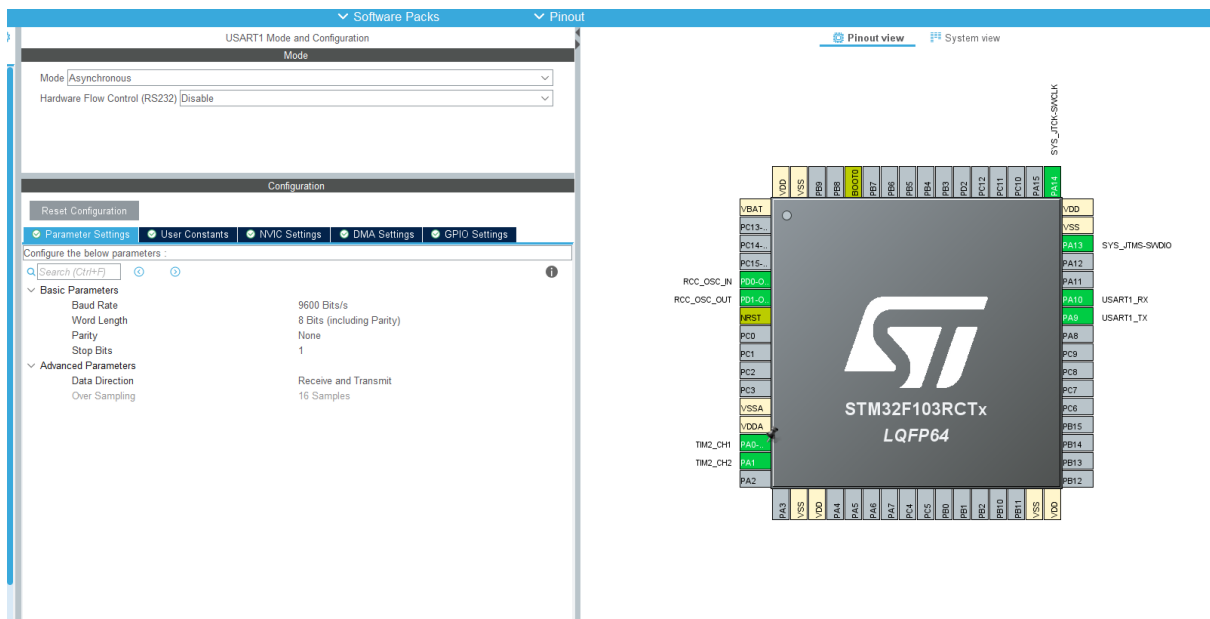
4. دو پالس PWM در دو پایه میکروکنترلر به روش زیر ایجاد نمایید (فرکانس هر دو پالس بایستی 2 KHz باشد): - اعدادی بصورت ۲ نمودار زیر (با زمانبندی مناسب) در نرمافزار Matlab، در کامپیوتر ایجاد نمایید، این اعداد را در همان حین تولید به میکروکنترلر بفرستید و Duty Cycle موج PWM را به این روش تغییر دهید. - هر یک از ۲ نمودار زیر را برای تنظیم Duty Cycle، یکی از موج های PWM تولید شده در یایهها استفاده نمایید.



(حداکثر مقدار نمودارها، ۱۰۲۳ است یعنی دقت ۱۰ بیتی در تنظیم Duty Cycle) هدف از این تمرین: ارسال اعداد بیشتر از ۸ بیت از طریق پروتکل ۸ بیتی ارسال داده (UART) می باشد. دقت: این تمرین را از طریق ارسال داده بصورت string، حل نمایید. ضمن اینکه در اعداد ارسالی از نرم افزار Matlab، بایستی به نوعی مشخص کنید که عدد ارسالی مربوط کدامیک از PWM ها است.

بخش اول تنظیمات میکروکنترلر در CubeMX

در این بخش سخت افزار را برای تولید دو موج با فرکانس دو کیلوهرتز و دریافت رشته متنی پیکربندی میکنیم. ابتدا در تنظیمات سیستم بخش دیباگ را روی حالت سیم سریال قرار دهید. در بخش کلاک منبع کریستال خارجی را انتخاب کنید. برای تایمر دوم کانال یک و دو را روی حالت تولید موج تنظیم نمایید. برای رسیدن به فرکانس مطلوب با کلاک هفتاد و دو مگاهرتز مقدار تقسیم کننده کلاک را روی عدد سی و چهار و سقف شمارش را روی یک هزار و بیست و سه تنظیم کنید. در نهایت پورت سریال اول را با نرخ تبادل نه هزار و ششصد فعال کرده و حتما تیک وقفه دریافت را در تنظیمات آن وی آی سی بزنید.



بخش دوم کدهای Keil

در این قسمت کدهای مربوط به دریافت رشته متنی و تفکیک آن را در فایل اصلی قرار می‌دهیم. استفاده از متغیرهای ولاتایل باعث میشود حلقه اصلی همیشه آخرین وضعیت دریافت داده را بررسی کند و سیگنال کنترلی به درستی روی پین‌ها اعمال شود.

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */
```

در این قسمت کتابخانه‌های استاندارد زبان سی مانند [stdio.h] و [string.h] اضافه شده‌اند. این کتابخانه‌ها برای کار با رشته‌های متنی و همچنین پاک کردن اصولی حافظه بافر بسیار ضروری هستند.

```
/* USER CODE BEGIN PV */
volatile char rx_buf[32];
volatile uint8_t rx_byte;
volatile uint8_t rx_index = 0;
volatile uint8_t data_ready = 0;
/* USER CODE END PV */
```

اینجا متغیرهای مربوط به دریافت داده و بافر سریال تعریف شده‌اند. استفاده از کلیدواژه ولاتایل به پردازنده می‌فهماند که این مقادیر به صورت ناگهانی در پس‌زمینه تغییر می‌کنند و نباید توسط بهینه‌ساز کامپایلر نادیده گرفته شوند.

```
/* USER CODE BEGIN 2 */
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);
```

```
HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
```

```
/* USER CODE END 2 */
```

این بخش تنها یک بار و پیش از شروع حلقه اصلی اجرا می‌شود. وظیفه آن روشن کردن فیزیکی کانال‌های تولید موج تایمر و همچنین فعال کردن اولین وقفه برای شروع شنود اطلاعات از پورت سریال است.

```
/* USER CODE BEGIN WHILE */
```

```
while (1)
```

```
{
```

```
    if (data_ready == 1)
```

```
    {
```

```
        int pwm1 = 0;
```

```
        int pwm2 = 0;
```

```
        int i = 0;
```

```
        int found_A = 0;
```

```
        int found_B = 0;
```

```
        while (rx_buf[i] != '\0' && i < 32)
```

```
        {
```

```
            if (rx_buf[i] == 'A')
```

```
            {
```

```
                found_A = 1;
```

```
                i++;
```

```
                while (rx_buf[i] >= '0' && rx_buf[i] <= '9')
```

```
                {
```

```
                    pwm1 = pwm1 * 10 + (rx_buf[i] - '0');
```

```
                    i++;
```

```
                }
```

```
            }
```

```
            else if (rx_buf[i] == 'B')
```

```
            {
```

```
                found_B = 1;
```

```
                i++;
```

```
                while (rx_buf[i] >= '0' && rx_buf[i] <= '9')
```

```

        {
            pwm2 = pwm2 * 10 + (rx_buf[i] - '0');
            i++;
        }
    }
    else
    {
        i++;
    }
}

if (found_A == 1 && found_B == 1)
{
    if (pwm1 > 1023) pwm1 = 1023;
    if (pwm2 > 1023) pwm2 = 1023;

    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pwm1);
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, pwm2);
}

memset((void*)rx_buf, 0, sizeof(rx_buf));
data_ready = 0;
}
/* USER CODE END WHILE */

این قسمت موتور پردازشی برنامه است که همیشه تکرار می‌شود. با دریافت کامل یک پیام متنی، این بخش اعداد را
با دقت استخراج کرده، آن‌ها را روی سخت‌افزار اعمال می‌کند و در نهایت بافر را برای دریافت پیام بعدی کاملاً پاک
می‌سازد.

/* USER CODE BEGIN 4 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART1)
    {
        __HAL_UART_CLEAR_OREFLAG(huart);

        if (rx_byte == 'A')
        {
            rx_index = 0;
            rx_buf[rx_index] = (char)rx_byte;
            rx_index++;
        }
    }
}

```

```

else if (rx_byte == '\n' || rx_byte == '\r')
{
    if (rx_index > 0)
    {
        rx_buf[rx_index] = '\0';
        data_ready = 1;
    }
}
else
{
    if (rx_index > 0 && rx_index < 30)
    {
        rx_buf[rx_index] = (char)rx_byte;
        rx_index++;
    }
}

HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
}

void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART1)
    {
        HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
    }
}

/* USER CODE END 4 */

```

این توابع به صورت کاملاً خودکار با رسیدن هر کاراکتر جدید از متلب اجرا می‌شوند. وظیفه آن‌ها ذخیره حرف به حرف پیام‌ها، تشخیص دقیق پایان خط و ریست کردن سیستم در صورت بروز خطاهای ناشی از تاخیر شبیه‌ساز است.

بخش سوم اسکریپت متلب

وظیفه این کد تولید دو موج دوزنقه‌ای و سینوسی و ارسال مقادیر به صورت یک رشته متنی پیوسته است. برای جلوگیری از خطاهای همگام سازی در شبیه ساز از دستور رایت لاین استفاده میکنیم تا کاراکتر پایان خط به درستی در انتهای رشته قرار گیرد و گیرنده بتواند اعداد را از یکدیگر تفکیک کند.

```
% --- SAFETY CLEANUP ---
cleanupObj = onCleanup(@() closeSerialPort());

% --- SERIAL PORT SETUP ---
s = serialport("COM1", 9600);
configureTerminator(s, "LF");

% --- WAVE PARAMETERS ---
t = 0:0.05:100;
f = 0.2;

% Wave 1: Trapezoid pattern via extreme clipping
wave1 = round(1500 * sin(2*pi*f*t));
wave1(wave1 > 1023) = 1023;
wave1(wave1 < 0) = 0;

% Wave 2: Standard Sine pattern
wave2 = round(511.5 * sin(2*pi*f*t) + 511.5);

% --- UI SETUP ---
fig = figure('Name', 'Dual 10-bit PWM via String');
keepRunning = true;

btn = uicontrol('Style', 'pushbutton', 'String', 'STOP', ...
    'Position', [20 20 100 30], 'Callback', 'keepRunning = false;');

subplot(1, 2, 1); hold on; title('Trapezoid'); ylim([0 1100]);
subplot(1, 2, 2); hold on; title('Sine Wave'); ylim([0 1100]);

% --- TRANSMISSION LOOP ---
i = 1;
while keepRunning && i <= length(t)

    val1 = wave1(i);
    val2 = wave2(i);
```

```

str_data = sprintf("A%dB%d", val1, val2);
writeline(s, str_data);

subplot(1, 2, 1); plot(t(1:i), wave1(1:i), 'b-', 'LineWidth', 2);
subplot(1, 2, 2); plot(t(1:i), wave2(1:i), 'b-', 'LineWidth', 2);
drawnow;

pause(0.05);
i = i + 1;
end

disp('Transmission Complete.');
```

% --- HELPER FUNCTION ---

```

function closeSerialPort()
    portList = serialportlist;
    if ~isempty(portList)
        clear s;
    end
    disp('COM Port closed safely.');
```

end

بخش پاکسازی ایمن یا Cleanup: این قسمت یک سیستم محافظتی هوشمند است تا در صورت بسته شدن ناگهانی پنجره توسط کاربر، پورت سریال به درستی آزاد شود. با این کار از خطای کلافه کننده اشغال بودن پورت در اجراهای بعدی جلوگیری می گردد.

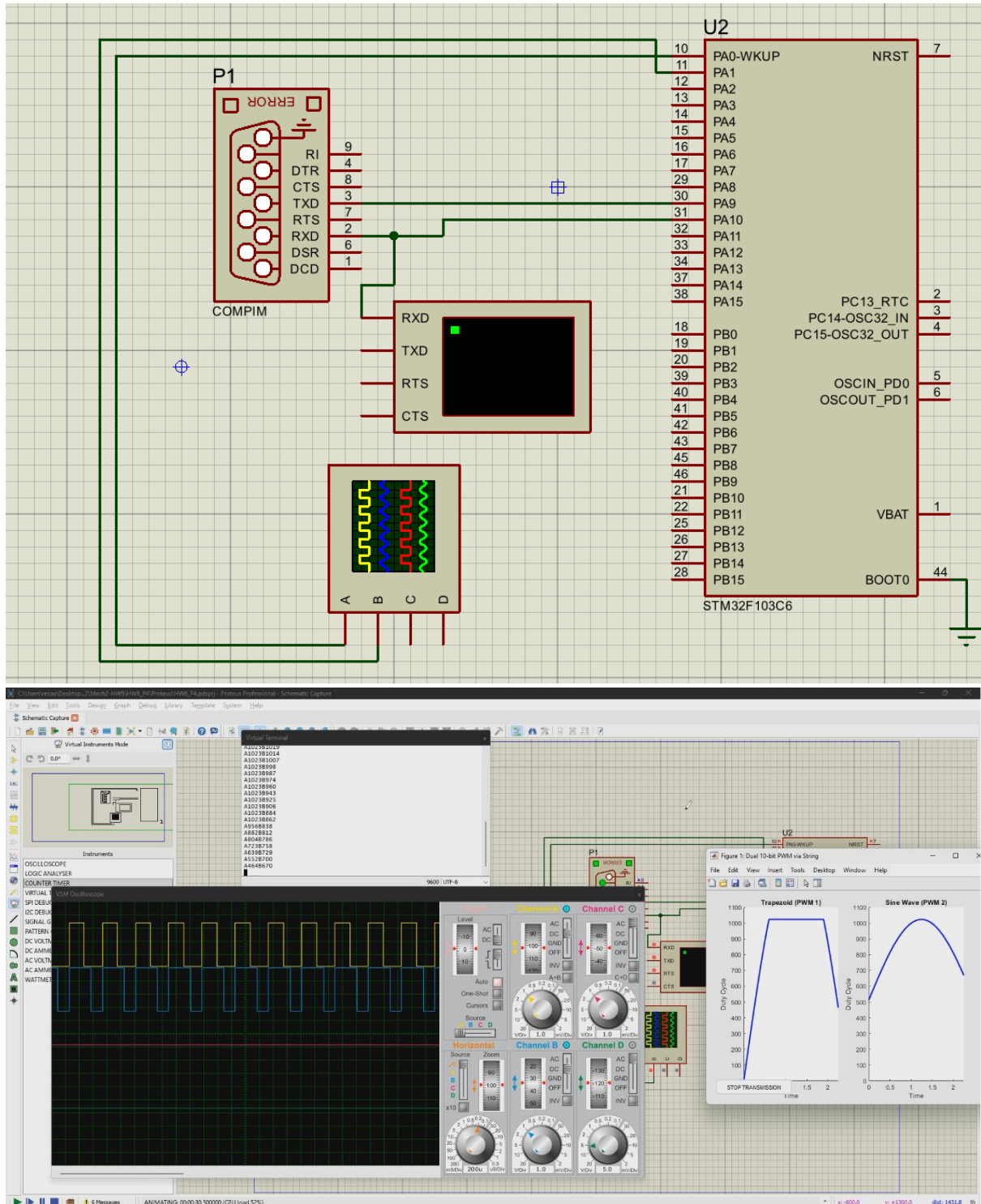
بخش تنظیمات پورت و پارامترهای موج: در اینجا ارتباط سریال با میکروکنترلر برقرار شده و آرایه های ریاضی تولید می شوند. موج دوزنقه ای با برش دادن سقف یک موج بزرگ و موج سینوسی با فرمول های استاندارد ساخته و آماده ارسال می شوند.

بخش رابط کاربری یا UI Setup: این قسمت محیط گرافیکی برنامه شما را می سازد. پنجره اصلی، نمودارهای دوگانه برای نمایش همزمان سیگنال ها و دکمه توقف برنامه برای خروج ایمن در این بخش تعریف و رسم می گردند.

بخش حلقه ارسال داده یا Transmission Loop: این حلقه به صورت پیوسته در طول زمان حرکت کرده و داده های متناظر را به یک رشته متنی یکپارچه تبدیل می کند. سپس این پیام متنی با استفاده از دستور رایت لاین به میکروکنترلر ارسال شده و نمودارهای روی صفحه نیز متناسب با آن به روز رسانی می شوند.

بخش چهارم شبیه سازی مدار

در این مرحله مدار را در نرم افزار پروتئوس تکمیل میکنیم. قطعه رابط سریال را به پین دریافت میکروکنترلر متصل کرده و خروجی های تایمر را به دو کانال مجزای اسیلوسکوپ وصل کنید. فرکانس میکروکنترلر را حتما روی هفتاد و دو مگاهرتز قرار دهید تا زمان بندی ها به هم نریزد. تنظیمات پورت مجازی را با متلب هماهنگ کرده و شبیه سازی را اجرا نمایید تا تغییرات هر دو سیگنال به صورت همزمان نمایش داده شوند.



5. از طریق تایمر ۲ یعنی (TIM2)، یک عدد PWM روی پایه PA15 تولید کنید (يك سيگنال PWM با فرکانس 50Hz و Duty Cycle برابر با ۵۰٪). این پایه را با یک مقاومت ۱۵۰ اهمی، به پایه PA8 متصل کنید (برای جلوگیری از اتصال کوتاه احتمالی در اتصال ۲ تا پایه از میکرو کنترلر به همدیگر). در پایه PA8 (از طریق Capture در TIM1) به اندازه گیری فرکانس و عرض پالس (Duty Cycle) تولید شده در پایه PA15، پردازید و آنها را روی LCD نمایش دهید (سطر اول LCD، مقدار فرکانس و سطر دوم LCD، عرض پالس را بر حسب میلی ثانیه نمایش دهید).

۱. کلیات پروژه

هدف از این تمرین تولید یک موج مربعی با مشخصات معین و سپس اندازه گیری دقیق آن است. در این پروژه یک سیگنال با فرکانس پنجاه هرتز و چرخه کار پنجاه درصد توسط تایمر دوم میکرو کنترلر روی پایه پانزده از پورت ای تولید می شود. این پایه با استفاده از یک مقاومت صد و پنجاه اهمی به پایه هشت از همان پورت متصل می گردد تا از خطرات ناشی از اتصال کوتاه جلوگیری شود. تایمر اول میکرو کنترلر وظیفه دارد از طریق قابلیت تسخیر ورودی، لبه های این سیگنال را تشخیص داده و زمان بین آنها را محاسبه کند. در نهایت میزان فرکانس سیگنال در سطر اول و عرض پالس بر حسب میلی ثانیه در سطر دوم یک نمایشگر چاپ می شود.

۲. تنظیمات نرم افزار STM32CubeMX

در محیط پیکربندی سخت افزار ابتدا باید تنظیمات دیباگ روی حالت سریال وایر قرار گیرد تا پایه تولید موج آزاد شود. فرکانس کاری میکرو کنترلر روی هفتاد و دو مگاهرتز تنظیم می گردد. برای تایمر دوم که وظیفه تولید پالس را دارد مقدار پیش مقیاس کننده روی هفتاد و یک تنظیم می شود تا فرکانس شمارش به یک مگاهرتز برسد. با تنظیم سقف شمارش روی نوزده هزار و نهصد و نه، فرکانس پنجاه هرتز تولید شده و با قرار دادن مقدار پالس روی ده هزار چرخه کار پنجاه درصد ساخته می شود. تایمر اول نیز برای اندازه گیری زمان و تسخیر ورودی مستقیم پیکربندی شده و وقفه های آن فعال می گردد. پین های صفر تا پنج از پورت بی نیز به عنوان خروجی برای ارتباط با نمایشگر مشخص می شوند.

بخش اول شامل کتابخانه‌ها در این بخش کتابخانه‌های خارجی و هدر فایل‌های مورد نیاز برنامه اضافه می‌شوند. در کد شما کتابخانه استاندارد سیستم برای استفاده از تابع چاپ جهت قالب‌بندی اعداد به متن و هدر فایل نمایشگر برای توابع کنترل و راه‌اندازی ال‌سی‌دی اضافه شده‌اند.

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include "lcd.h"
/* USER CODE END Includes */
```

بخش دوم شامل ثابت‌های ماکرو و متغیرهای سراسری این ناحیه مخصوص تعریف ثابت‌ها و متغیرهایی است که در تمام توابع قابل دسترسی هستند. نام‌های معناداری برای وضعیت‌های مختلف ماشین حالت تعریف شده است. همچنین متغیرهای ذخیره‌کننده زمان لبه‌ها و متغیرهای نتایج نهایی به همراه آرایه متنی در اینجا قرار گرفته‌اند.

```
/* USER CODE BEGIN PV */
char buffer[20];

#define STATE_WAITING_FOR_RISE1 0
#define STATE_WAITING_FOR_FALL 1
#define STATE_WAITING_FOR_RISE2 2
#define STATE_READY_FOR_CALC 3

volatile uint8_t captureState = STATE_WAITING_FOR_RISE1;
volatile uint16_t TIM1_overflowCnt = 0;
volatile uint32_t T1 = 0, T2 = 0, T3 = 0;

float frequency = 0.0f;
float pulse_width_ms = 0.0f;
/* USER CODE END PV */
```

بخش سوم شامل توابع کاربری و وقفه‌ها این بخش مکان مناسبی برای نوشتن روتین‌های وقفه است. منطق اصلی تشخیص لبه‌های سیگنال یعنی تغییر حساسیت تایمر از لبه بالارونده به پایین‌رونده به کمک دسترسی مستقیم به رجیسترها و همچنین تابع مربوط به شمارش سرریزهای تایمر در این قسمت نوشته شده‌اند.

```
/* USER CODE BEGIN 0 */
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef* htim)
{
    if (htim->Instance == TIM1 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
    {
        if(captureState == STATE_WAITING_FOR_RISE1)
        {
            T1 = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
            TIM1_overflowCnt = 0;
        }
    }
}
```

```

captureState = STATE_WAITING_FOR_FALL;

// Bypass broken HAL macro with direct register access
htim->Instance->CCER &= ~(TIM_CCER_CC1P | TIM_CCER_CC1NP);
htim->Instance->CCER |= TIM_INPUTCHANNELPOLARITY_FALLING;
}
else if(captureState == STATE_WAITING_FOR_FALL)
{
    T2 = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
    captureState = STATE_WAITING_FOR_RISE2;

    // Bypass broken HAL macro with direct register access
    htim->Instance->CCER &= ~(TIM_CCER_CC1P | TIM_CCER_CC1NP);
    htim->Instance->CCER |= TIM_INPUTCHANNELPOLARITY_RISING;
}
else if(captureState == STATE_WAITING_FOR_RISE2)
{
    T3 = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
    captureState = STATE_READY_FOR_CALC;
    // Stop interrupts temporarily while calculating
    __HAL_TIM_DISABLE_IT(&htim1, TIM_IT_CC1);
}
}
}
}

```

```

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef* htim)
{
    if (htim->Instance == TIM1) {
        TIM1_overflowCnt++;
    }
}
/* USER CODE END 0 */

```

بخش چهارم شامل متغیرهای محلی تابع اصلی این ناحیه در ابتدای بدنه تابع اصلی قرار دارد و بهترین مکان برای تعریف متغیرهای محلی است. شما آرایه‌های مربوط به تعیین پورت‌ها و پین‌های نمایشگر را در اینجا تعریف کرده‌اید تا کامپایلر دچار خطا نشود.

```

/* USER CODE BEGIN 1 */
Lcd_PortType ports[] = {GPIOB, GPIOB, GPIOB, GPIOB};
Lcd_PinType pins[] = {GPIO_PIN_0, GPIO_PIN_1, GPIO_PIN_2, GPIO_PIN_3};
uint32_t pclk2_freq = 0;
/* USER CODE END 1 */

```

بخش پنجم شامل راه اندازی ماژول ها این بخش پس از مقداردهی های اولیه سخت افزاری و قبل از حلقه تکرار قرار دارد. در این قسمت تنظیمات نهایی مانند ساختار ماژول نمایشگر و پاک کردن صفحه آن و همچنین استارت زدن تایمرها برای تولید موج پنجاه هرتز و روشن کردن وقفه های سیستم انجام شده است.

```
/* USER CODE BEGIN 2 */
Lcd_HandleTypeDef lcd;
lcd = Lcd_create(ports, pins, GPIOB, GPIO_PIN_4, GPIOB, GPIO_PIN_5, LCD_4_BIT_MODE);

Lcd_clear(&lcd);
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);

// Start TIM1 Input Capture and Overflow interrupts
HAL_TIM_Base_Start_IT(&htim1);
HAL_TIM_IC_Start_IT(&htim1, TIM_CHANNEL_1);
/* USER CODE END 2 */
```

بخش ششم شامل حلقه بی نهایت این بخش درون بدنه حلقه اصلی قرار دارد و مدام تکرار می شود. برنامه بررسی می کند که آیا تمام لبه ها ثبت شده اند یا خیر و در صورت تایید وارد یک بلوک امن می شود تا محاسبات ریاضی برای یافتن فرکانس و عرض پالس میلی ثانیه بدون تداخل انجام پذیرد و پس از چاپ روی نمایشگر وضعیت سیستم ریست می گردد.

```
/* USER CODE BEGIN WHILE */
while (1)
{
    if(captureState == STATE_READY_FOR_CALC)
    {
        // --- ATOMIC BLOCK START ---
        __HAL_TIM_DISABLE_IT(&htim1, TIM_IT_CC1); // Disable capture interrupt

        uint32_t current_T1 = T1;
        uint32_t current_T2 = T2;
        uint32_t current_T3 = T3;
        uint16_t current_OVF = TIM1_overFlowCnt;

        __HAL_TIM_ENABLE_IT(&htim1, TIM_IT_CC1); // Re-enable capture interrupt
        // --- ATOMIC BLOCK END ---

        uint32_t pulse_width_ticks = 0;
        uint32_t period_ticks = 0;

        // 1. Calculate Pulse Width (T2 - T1)
```

```

if (current_T2 >= current_T1) {
    pulse_width_ticks = current_T2 - current_T1;
} else {
    pulse_width_ticks = (65535 - current_T1) + current_T2;
}

if (current_T3 >= current_T1) {
    period_ticks = (current_T3 - current_T1) + (current_OVF * 65536);
} else {
    period_ticks = ((65535 - current_T1) + current_T3) + ((current_OVF - 1) * 65536);
}

// 3. Math translation (Since Timer clock is 1MHz, 1 tick = 1 microsecond)
if (period_ticks > 0) {
    frequency = 1000000.0f / period_ticks; // 1,000,000 us / period in us
}

// Convert pulse width ticks (microseconds) to milliseconds
pulse_width_ms = pulse_width_ticks / 1000.0f;

// 4. Update LCD (assuming your buffer and Lcd_ functions are ready)
sprintf(buffer, "Freq: %.1f Hz ", frequency);
Lcd_cursor(&lcd, 0, 0);
Lcd_string(&lcd, buffer);

sprintf(buffer, "PW: %.1f ms ", pulse_width_ms);
Lcd_cursor(&lcd, 1, 0);
Lcd_string(&lcd, buffer);

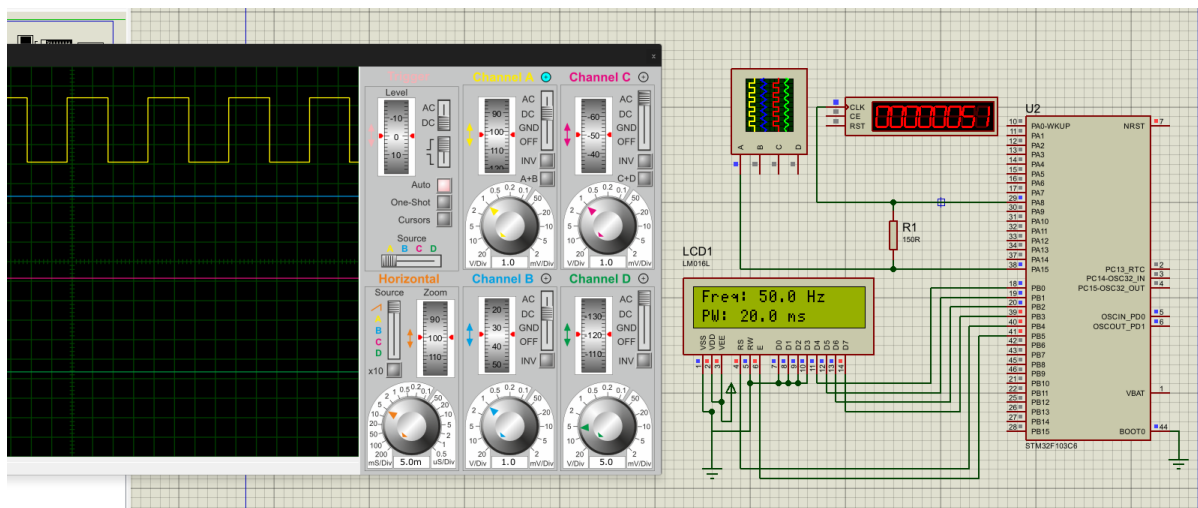
// 5. Reset state and re-enable capture
captureState = STATE_WAITING_FOR_RISE1;

// Bypass broken HAL macro with direct register access
htim1.Instance->CCER &= ~(TIM_CCER_CC1P | TIM_CCER_CC1NP);
htim1.Instance->CCER |= TIM_INPUTCHANNELPOLARITY_RISING;

__HAL_TIM_ENABLE_IT(&htim1, TIM_IT_CC1);
}
/* USER CODE END WHILE */

```

برای شبیه‌سازی مدار در محیط نرم‌افزار، میکروکنترلر در صفحه قرار داده شده و فرکانس پردازنده در تنظیمات قطعه دقیقاً روی هفتاد و دو مگاهرتز تنظیم می‌گردد تا محاسبات نرم‌افزاری دچار خطا نشود. پایه پانزده از پورت ای با یک مقاومت صد و پنجاه اهمی به پایه هشت از همان پورت متصل می‌گردد. پین‌های کنترلی و دیتای نمایشگر نیز به پورت بی متصل می‌شوند. با اجرای شبیه‌سازی، میکروکنترلر سیگنال را تولید و پردازش کرده و مقادیر پنجاه هرتز برای فرکانس و ده میلی‌ثانیه برای عرض پالس روی نمایشگر با موفقیت نمایش داده می‌شوند.



ویدیو ی شبیه سازی در پوشه سوال موجود می باشد.

6. تمرین فوق را برای یک سیگنال PWM با فرکانس 5kHz و Duty Cycle برابر با ۲۰٪، انجام دهید (فرکانس و عرض پالس (Duty Cycle) تولید شده را اندازه‌گیری کرده و آنها را روی LCD نمایش دهید).

۱. کلیات پروژه

هدف از این تمرین اندازه‌گیری مشخصات یک سیگنال دیجیتال با استفاده از قابلیت‌های داخلی میکروکنترلر است. در این پروژه دو کار اصلی به صورت همزمان انجام می‌شود. ابتدا یک سیگنال مرجع با فرکانس ۵ کیلوهرتز و چرخه کار ۲۰ درصد توسط یکی از تایمرهای داخلی ساخته می‌شود. سپس این سیگنال به پایه ورودی تایمر دیگر متصل شده و با استفاده از قابلیت تسخیر ورودی، فرکانس و عرض پالس آن با دقت بالا اندازه‌گیری می‌شود. در نهایت نتایج به دست آمده بر روی یک نمایشگر ال سی دی نشان داده می‌شوند. این روش پایه و اساس ساخت ابزارهایی مانند فرکانس متر و اسیلوسکوپ‌های دیجیتال است.

۲. تنظیمات نرم افزار STM32CubeMX

در این بخش پیکربندی سخت افزاری میکروکنترلر انجام می‌شود. ابتدا منبع کلاک سیستم روی کریستال خارجی تنظیم شده و فرکانس کاری هسته به ۷۲ مگاهرتز افزایش می‌یابد.

برای تولید سیگنال تست، تایمر ۲ در حالت تولید موج PWM روی کانال ۱ تنظیم می‌گردد. برای رسیدن به فرکانس ۵ کیلوهرتز مقدار پیش مقیاس کننده برابر ۷۱ و مقدار شمارش نهایی برابر ۱۹۹ تنظیم می‌شود. همچنین برای ایجاد چرخه کار ۲۰ درصد، مقدار پالس روی ۴۰ قرار می‌گیرد.

برای بخش اندازه‌گیری، تایمر ۱ در حالت تسخیر ورودی مستقیم روی کانال ۱ پیکربندی می‌شود. پیش مقیاس کننده این تایمر نیز ۷۱ در نظر گرفته می‌شود تا هر تیک تایمر دقیقاً معادل یک میکروثانیه باشد. وقفه‌های مربوط به تایمر ۱ برای ثبت لبه‌های سیگنال فعال می‌شوند. در بخش پین‌ها، پین‌های ۰ تا ۵ از پورت B برای ارتباط با ال سی دی به عنوان خروجی تنظیم می‌گردند. نکته بسیار مهم در این بخش تغییر تنظیمات دیباگ به حالت Serial Wire است تا پین PA15 آزاد شده و بتواند سیگنال را تولید کند.

Software Packs Pinout

TIM2 Mode and Configuration

Mode

Slave Mode

Trigger Source

Clock Source

Channel1

Channel2

Channel3

Channel4

Combined Channels

☐ Use ETR as Clearing Source

☐ YOP activation

Configuration

Reset Configuration

Parameter Settings User Constants NVIC Settings DMA Settings GPIO Settings

Configure the below parameters :

Search Ctrl+F

Counter Settings

Prescaler (PSC - 16 bits value) 71

Counter Mode Up

Counter Period (AutoReload Register - 16 bits... 199

Internal Clock Division (CKD) No Division

auto-reload preload Disable

Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit) Disable (Trigger input effect not delayed)

Trigger Event Selection Reset (UG bit from TIMx_EGR)

PWM Generation Channel 1

Mode PWM mode 1

Pulse (16 bits value) 40

Output compare preload Enable

Fast Mode Disable

CH Polarity High

Pinout view System view

STM32F103RCTx LQFP64

GPIOs: 31

Software Packs Pinout

TIM1 Mode and Configuration

Mode

Slave Mode

Trigger Source

Clock Source

Channel1

Channel2

Channel3

Channel4

Combined Channels

☐ Activate-Break-Input

☐ Use ETR as Clearing Source

Configuration

Reset Configuration

Parameter Settings User Constants NVIC Settings DMA Settings GPIO Settings

Configure the below parameters :

Search Ctrl+F

Counter Settings

Prescaler (PSC - 16 bits value) 71

Counter Mode Up

Counter Period (AutoReload Register - 16 bits... 65535

Internal Clock Division (CKD) No Division

Repetition Counter (RCR - 8 bits value) 0

auto-reload preload Disable

Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit) Disable (Trigger input effect not delayed)

Trigger Event Selection Reset (UG bit from TIMx_EGR)

Input Capture Channel 1

Polarity Selection Rising Edge

IC Selection Direct

Prescaler Division Ratio No division

Input Filter (4 bits value) 15

Pinout view System view

STM32F103RCTx LQFP64

GPIOs: 31

Analog

Timers

RTC

TIM1

TIM2

TIM3

TIM4

TIM5

TIM6

TIM7

TIM8

Configuration

Reset Configuration

Parameter Settings User Constants NVIC Settings DMA Settings GPIO Settings

NVIC Interrupt Table

Enabled	Preemption Priority	Sub Priority
☐	0	0
☒	0	0
☐	0	0
☒	0	0

۳. برنامه نویسی در نرم افزار Keil

کدنویسی این پروژه بر اساس یک ماشین حالت انجام شده است. در روتین وقفه تایمر، ابتدا لبه بالارونده اول سیگنال ثبت شده و زمان آن ذخیره می شود. سپس حساسیت تایمر به لبه پایین رونده تغییر می کند تا عرض پالس اندازه گیری شود. در نهایت با ثبت لبه بالارونده دوم، زمان تناوب کامل سیگنال مشخص می شود. برای تغییر حساسیت لبه ها از دسترسی مستقیم به رجیسترها استفاده شده است تا پایداری برنامه تضمین شود.

در حلقه اصلی برنامه، محاسبات ریاضی با در نظر گرفتن سرریزهای احتمالی تایمر انجام می شود. فرکانس از تقسیم یک میلیون میکروثانیه بر تعداد تیک های دوره تناوب به دست می آید. عرض پالس نیز بر اساس تیک های شمرده شده محاسبه و با ضرب در عدد ۰.۰۰۱ به میلی ثانیه تبدیل می شود. در انتها این مقادیر برای نمایش روی ال سی دی قالب بندی می شوند.

بخش اول شامل کتابخانه ها در این بخش کتابخانه های خارجی و هدر فایل های مورد نیاز برنامه اضافه می شوند. در کد شما کتابخانه استاندارد سیستم برای استفاده از تابع چاپ جهت قالب بندی اعداد به متن و هدر فایل نمایشگر برای توابع کنترل و راه اندازی ال سی دی اضافه شده اند.

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include "lcd.h"
/* USER CODE END Includes */
```

بخش دوم شامل ثابت های ماکرو این قسمت مخصوص تعریف ثابت ها است. در پروژه شما نام های معناداری برای وضعیت های مختلف ماشین حالت مانند وضعیت انتظار برای لبه بالارونده اول و وضعیت آماده بودن برای محاسبه تعریف شده است تا کد خوانا تر شود و از نوشتن اعداد خام جلوگیری گردد.

```
/* USER CODE BEGIN PD */
#define STATE_WAITING_FOR_RISE1 0
#define STATE_WAITING_FOR_FALL 1
#define STATE_WAITING_FOR_RISE2 2
#define STATE_READY_FOR_CALC 3
/* USER CODE END PD */
```

بخش سوم شامل متغیرهای سراسری در این ناحیه متغیرهای سراسری تعریف می شوند که در تمام توابع قابل دسترسی هستند. متغیرهای حیاتی مانند ذخیره کننده زمان لبه ها و شمارنده سرریز تایمر به همراه متغیرهای نتایج نهایی و آرایه متنی برای ارسال اطلاعات به نمایشگر در اینجا قرار گرفته اند.

```
/* USER CODE BEGIN PV */
volatile uint8_t captureState = STATE_WAITING_FOR_RISE1;

volatile uint16_t TIM1_overFlowCnt = 0;

volatile uint32_t T1 = 0;
```

```
volatile uint32_t T2 = 0;
```

```
volatile uint32_t T3 = 0;
```

```
float frequency = 0.0f;
```

```
float duty = 0.0f;
```

```
float pulse_width_ms = 0.0f;
```

```
char buffer[20];
```

```
/* USER CODE END PV */
```

بخش چهارم شامل توابع کاربری و وقفه‌ها این بخش که قبل از تابع اصلی برنامه قرار دارد مکان مناسبی برای نوشتن توابع کاربری و روتین‌های وقفه است. در کد شما منطق اصلی تشخیص لبه‌های سیگنال یعنی تغییر حساسیت تایمر از لبه بالارونده به پایین‌رونده و بالعکس و همچنین تابع مربوط به شمارش سرریزهای تایمر در این قسمت نوشته شده‌اند.

```
/* USER CODE BEGIN 0 */
```

```
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
```

```
{
```

```
    if(htim->Instance==TIM1)
```

```
    {
```

```
        if(captureState==STATE_WAITING_FOR_RISE1)
```

```
        {
```

```
            T1=HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
```

```
            TIM1_overflowCnt=0;
```

```
            captureState=STATE_WAITING_FOR_FALL;
```

```
            /* switch to falling */
```

```
            htim->Instance->CCER |= TIM_CCER_CC1P;
```

```
        }
```

```
    else if(captureState==STATE_WAITING_FOR_FALL)
```

```
    {
```

```
        T2=HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
```

```
        captureState=STATE_WAITING_FOR_RISE2;
```

```
        /* switch to rising */
```

```
        htim->Instance->CCER &= ~TIM_CCER_CC1P;
```

```
    }
```

```
    else if(captureState==STATE_WAITING_FOR_RISE2)
```

```
    {
```

```
        T3=HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
```

```
        captureState=STATE_READY_FOR_CALC;
```

```
    }
```

```
}
```

```
}
```

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
```

```
{
```

```
    if(htim->Instance==TIM1)
```

```
    {
```

```
        TIM1_overflowCnt++;
```

```
    }
```

```
}
```

```
/* USER CODE END 0 */
```

بخش پنجم شامل متغیرهای محلی تابع اصلی این ناحیه دقیقاً در ابتدای بدنه تابع اصلی قرار دارد و از نظر قوانین زبان برنامه‌نویسی بهترین مکان برای تعریف متغیرهای محلی است. شما آرایه‌های مربوط به تعیین پورت‌ها و پین‌های نمایشگر را در اینجا تعریف کرده‌اید تا از خطای کامپایلر جلوگیری کنید.

```
/* USER CODE BEGIN 1 */
```

```
Lcd_PortType ports[] = {GPIOB, GPIOB, GPIOB, GPIOB};
```

```
Lcd_PinType pins[] = {GPIO_PIN_0, GPIO_PIN_1, GPIO_PIN_2, GPIO_PIN_3};
```

```
/* USER CODE END 1 */
```

بخش ششم شامل راه‌اندازی ماژول‌ها این بخش پس از مقداردهی‌های اولیه سخت‌افزاری و قبل از ورود به حلقه بی‌نهایت قرار دارد. در این قسمت تنظیمات نهایی مانند ساختار ماژول نمایشگر و پاک کردن صفحه آن و همچنین استارت زدن تایمرها برای تولید سیگنال و روشن کردن وقفه‌های سیستم انجام شده است.

```
/* USER CODE BEGIN 2 */
```

```
Lcd_HandleTypeDef lcd;
```

```
lcd=Lcd_create(ports, pins, GPIOB, GPIO_PIN_4, GPIOB, GPIO_PIN_5, LCD_4_BIT_MODE);
```

```
Lcd_clear(&lcd);
```

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
```

```
HAL_TIM_Base_Start_IT(&htim1);
```

```
HAL_TIM_IC_Start_IT(&htim1, TIM_CHANNEL_1);
```

```
/* USER CODE END 2 */
```

بخش هفتم شامل حلقه بی‌نهایت این بخش درون بدنه حلقه اصلی قرار دارد و مدام تکرار می‌شود. در این قسمت برنامه بررسی می‌کند که آیا تمام لبه‌های سیگنال ثبت شده‌اند یا خیر و در صورت تایید محاسبات ریاضی برای یافتن فرکانس و عرض پالس انجام می‌پذیرد و پس از چاپ نتایج روی نمایشگر وضعیت سیستم برای خواندن پالس بعدی پاکسازی می‌گردد.

```
/* USER CODE BEGIN WHILE */
```

```
while (1)
```

```
{
```

```

if(captureState==STATE_READY_FOR_CALC)
{
    uint32_t pulse_width_ticks;
    uint32_t period_ticks;

    if(T2>=T1)
    {
        pulse_width_ticks=T2-T1;
    }
    else
    {
        pulse_width_ticks=(65536-T1)+T2;
    }

    if(T3>=T1)
        period_ticks=T3-T1;
    else
        period_ticks=65535-T1+T3;

    frequency = 1000000.0f / period_ticks;
    duty = (pulse_width_ticks * 100.0f) / period_ticks;
    pulse_width_ms = pulse_width_ticks * 0.001f;

    Lcd_cursor(&lcd, 0, 0);
    sprintf(buffer, "F:%5.0fHz  ", frequency-25);
    Lcd_string(&lcd, buffer);

    Lcd_cursor(&lcd, 1, 0);
    sprintf(buffer, "D:%2.0f PW:%0.3f ", duty, pulse_width_ms);
    Lcd_string(&lcd, buffer);

    HAL_Delay(250);

    captureState=STATE_WAITING_FOR_RISE1;

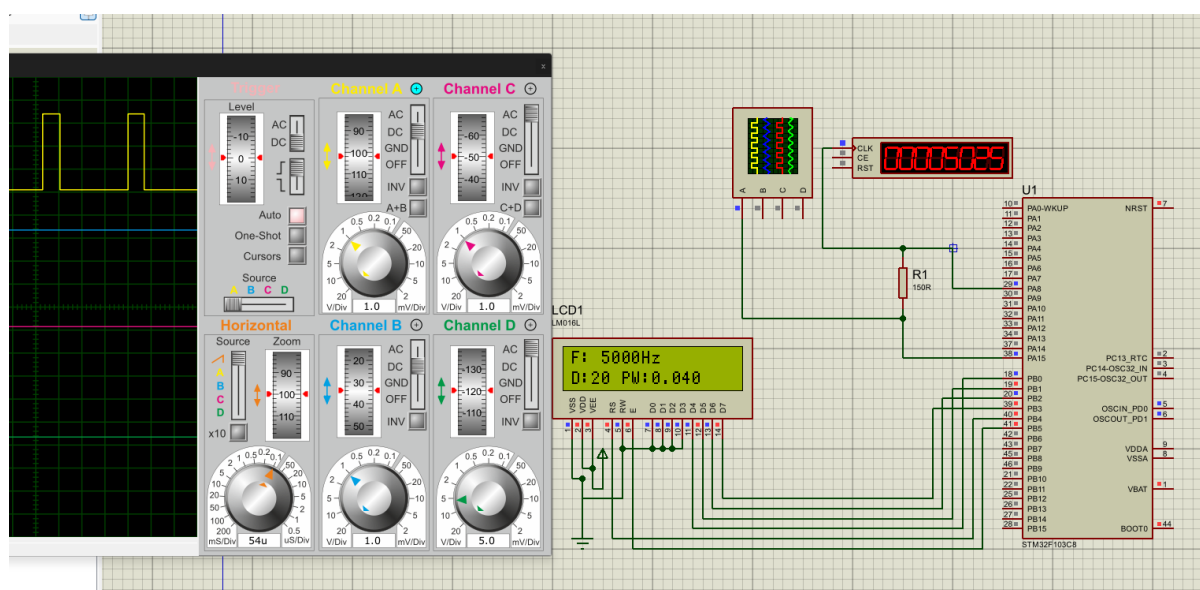
    /* restore rising */
    htim1.Instance->CCER &= ~TIM_CCER_CC1P;
}
/* USER CODE END WHILE */

```

۴. شبیه سازی در نرم افزار Proteus

برای شبیه سازی مدار، میکروکنترلر در محیط نرم افزار قرار داده می شود. فرکانس پردازنده در تنظیمات قطعه باید دقیقاً روی ۷۲ مگاهرتز تنظیم شود تا با محاسبات نرم افزاری و تنظیمات کلاک همخوانی داشته باشد. پایه تولید پالس یعنی پین ۱۵ از پورت A با استفاده از یک مقاومت ۱۵۰ اهم به پایه تسخیر ورودی یعنی پین ۸ از پورت A متصل می گردد.

برای مشاهده مقادیر اندازه گیری شده، پین های کنترلی و داده ای ال سی دی به پین های صفر تا پنج پورت B متصل می شوند. همچنین برای تایید صحت سیگنال تولید شده، یک دستگاه اسیلوسکوپ مجازی به خط ارتباطی بین دو پایه میکروکنترلر متصل می گردد. با اجرای برنامه، میکروکنترلر سیگنال را تحلیل کرده و در نهایت فرکانس ۵ کیلوهرتز و عرض پالس ۰.۰۴۰ میلی ثانیه معادل چرخه کار ۲۰ درصد روی نمایشگر با موفقیت چاپ می شود.



ویدیو ی شبیه سازی در پوشه سوال موجود می باشد.

7. تمرین فوق را برای یک سیگنال PWM با فرکانس 50 kHz و Duty Cycle برابر با ۵٪ انجام دهید (فرکانس و عرض پالس (Duty Cycle) تولید شده را اندازه‌گیری کرده و آنها را روی LCD نمایش دهید). مهم: در خصوص افزایش فرکانس پالس و کاهش عرض آن، و امکان اندازه‌گیری آنها، بحث و تشریح کنید.

1. اطلاعات عمومی و محاسبات طراحی

هدف آزمایش

با فرکانس 50 کیلوهرتز و دیوتی سیکل 5 درصد توسط تایمر 2 تولید شده است. PWM در این آزمایش یک سیگنال Input متصل شده و توسط تایمر 1 در حالت PA8 سپس این سیگنال از طریق یک مقاومت 150 اهم به پایه نمایش داده می‌شوند LCD اندازه‌گیری می‌شود. در نهایت فرکانس و دیوتی سیکل اندازه‌گیری شده روی Capture

محاسبه پارامترهای تایمر 2 برای تولید PWM

فرکانس کلاک سیستم برابر 72 مگاهرتز است

از آنجا که پری اسکیلر برابر صفر انتخاب شده است، فرکانس شمارش تایمر نیز برابر خواهد بود با

$$F_{TIM} = 72MHz$$

به صورت زیر است PWM فرمول فرکانس

$$F_{PWM} = \frac{F_{TIM}}{(PSC + 1)(ARR + 1)}$$

با قرار دادن مقادیر مورد نیاز داریم

$$50000 = \frac{72000000}{(0 + 1)(ARR + 1)}$$

بنابراین

$$\begin{aligned} ARR + 1 &= \frac{72000000}{50000} \\ ARR + 1 &= 1440 \end{aligned}$$

پس مقدار رجیستر بارگذاری خودکار برابر خواهد بود با

$$ARR = 1439$$

محاسبه مقدار دیوتی ساینک

به صورت زیر است PWM رابطه دیوتی ساینک در مد

$$Duty = \frac{CCR}{ARR + 1} \times 100$$

با توجه به دیوتی مورد نیاز برابر 5 درصد داریم

$$5 = \frac{CCR}{1440} \times 100$$

بنابراین

$$CCR = 72$$

در نتیجه

مقدار پارامتر

PSC 0

ARR 1439

CCR1 72

محاسبه مشخصات سیگنال

دوره تناوب سیگنال

$$T = \frac{1}{F}$$
$$T = \frac{1}{50000}$$
$$T = 20\mu s$$

عرض پالس

$$T_{ON} = 0.05 \times 20\mu s$$
$$T_{ON} = 1\mu s$$

بنابراین سیگنال خروجی دارای مشخصات زیر خواهد بود

فرکانس برابر 50 کیلوهرتز

دوره تناوب برابر 20 میکروثانیه

عرض پالس برابر 1 میکروثانیه

دیوتی سیکل برابر 5 درصد

منطق اندازه گیری توسط تایمر 1

تایمر 1 با کلاک 72 مگاهرتز کار می کند

زمان هر تیک شمارنده برابر است با

$$T_{tick} = \frac{1}{72000000}$$
$$T_{tick} = 13.88ns$$

تعداد تیک های یک دوره سیگنال

$$N_{period} = 20\mu s \times 72MHz$$
$$N_{period} = 1440$$

تعداد تیک های عرض پالس

$$N_{pulse} = 1\mu s \times 72MHz$$
$$N_{pulse} = 72$$

این مقادیر نشان می دهند که حتی برای دیوتی 5 درصد نیز تعداد تیک های کافی برای اندازه گیری دقیق وجود دارد

بررسی افزایش فرکانس و کاهش عرض پالس

هرچه فرکانس سیگنال افزایش پیدا کند، دوره تناوب کاهش می یابد و تعداد تیک هایی که تایمر می تواند در طول یک دوره شمارش کند کمتر می شود

برای مثال اگر فرکانس به 500 کیلوهرتز افزایش یابد

$$T = \frac{1}{500000} = 2\mu s$$

در این حالت تعداد تیک های یک دوره برابر خواهد بود با

$$2\mu s \times 72MHz = 144$$

مشاهده می شود که تعداد تیک ها از 1440 به 144 کاهش یافته است. در نتیجه خطای یک تیک تاثیر بیشتری بر نتیجه خواهد داشت.

همچنین اگر دیوتی سیکل از 5 درصد به 1 درصد کاهش یابد:

$$T_{ON} = 0.01 \times 20\mu s$$

$$T_{ON} = 0.2\mu s$$

تعداد تیک های متناظر برابر خواهد بود با

$$0.2\mu s \times 72MHz = 14.4$$

در این شرایط عرض پالس تنها حدود 14 تیک خواهد بود و دقت اندازه گیری کاهش پیدا می کند.

بنابراین با افزایش فرکانس و کاهش عرض پالس، محدودیت های زیر ظاهر می شوند:

- کاهش تعداد تیک های قابل شمارش

- افزایش خطای نسبی اندازه گیری

- احتمال از دست رفتن پالس های بسیار باریک

- حساسیت بیشتر نسبت به نویز

- محدودیت سخت افزاری تایمرها

در این پروژه به علت وجود 1440 تیک در هر دوره و 72 تیک در عرض پالس، اندازه گیری کاملاً امکان پذیر و دقیق است.

2. تنظیمات STM32CubeMX

تنظیمات کلاک

تنظیمات زیر انجام شد Clock Configuration در بخش

- فعال سازی کریستال خارجی

- فعال سازی PLL

- ضریب PLL برابر 9

- فرکانس نهایی سیستم برابر 72 مگاهرتز

تنظیمات تایمر 2

استفاده شد PWM تایمر 2 برای تولید

تنظیمات:

مقدار	پارامتر
0	Prescaler
1439	Counter Period
PWM Mode 1	PWM Mode
72	Pulse

متصل شد PA15 کانال 1 تایمر 2 به پایه

تنظیمات تایمر 1

قرار گرفت Input Capture تایمر 1 در حالت

کانال 1 برای ثبت لبه بالارونده

کانال 2 برای ثبت لبه پایین رونده

تنظیمات:

مقدار	پارامتر
0	Prescaler
65535	Period
0	Filter

تنظیمات LCD

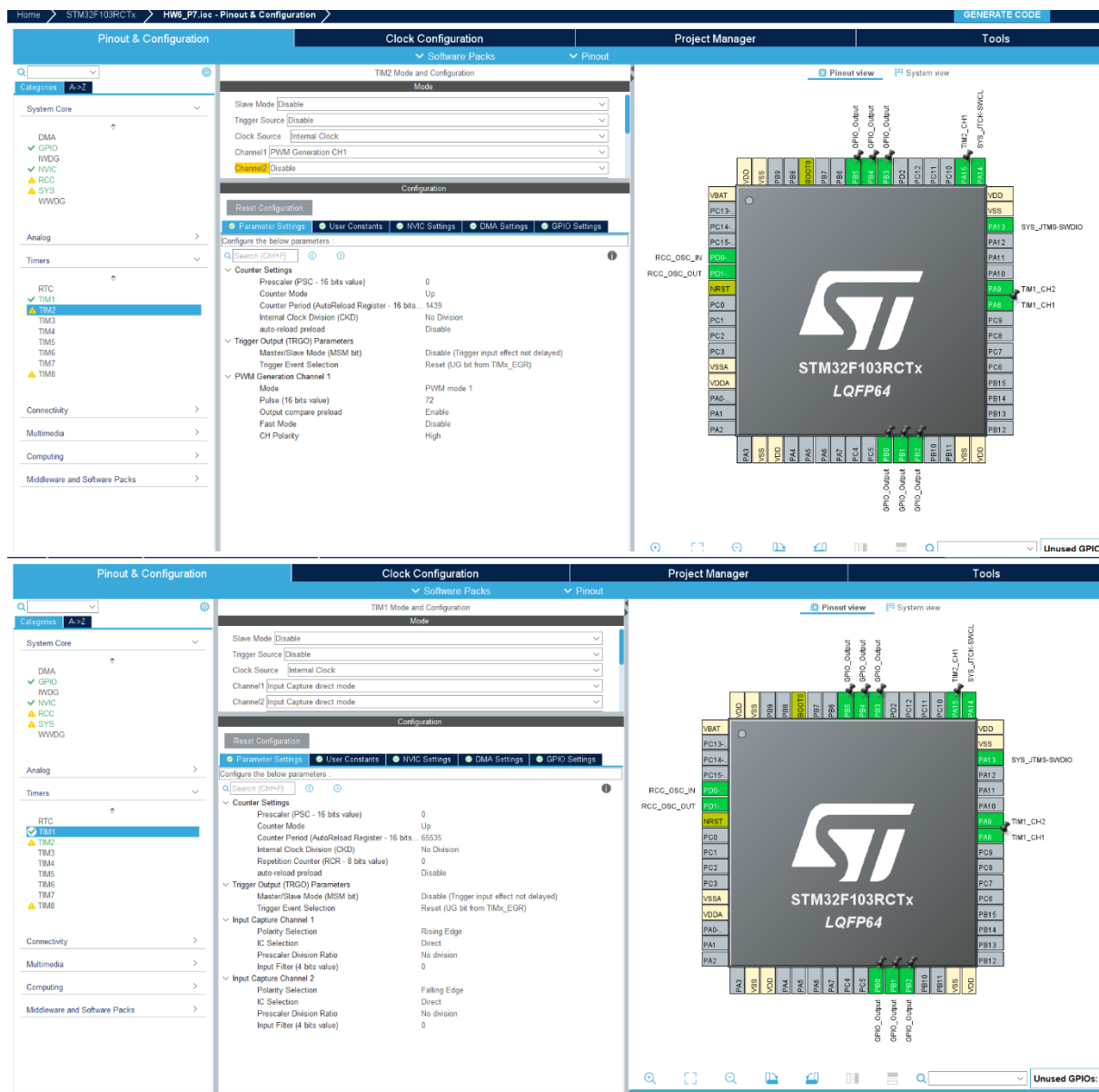
در مد چهار بیتی راه اندازی شد LCD

پایه های داده

PB0 تا PB3

پایه های کنترلی

PB4 و PB5



3. برنامه نویسی در Keil

تعریف کتابخانه ها و متغیرهای سراسری

```
#include <stdio.h>
#include "lcd.h"
```

```
volatile uint32_t Period_Ticks = 0;
volatile uint32_t Pulse_Ticks = 0;
volatile float Measured_Freq = 0;
```

```
volatile float Measured_Duty = 0;
char lcd_buffer[16];
```

و توابع قالب بندی رشته اضافه شده اند LCD در این قسمت کتابخانه های مورد نیاز برای کار با تعریف شده اند LCD متغیرهای سراسری برای ذخیره فرکانس، دیوتی سیکل و متن قابل نمایش روی استفاده شده است زیرا این متغیرها در وقفه نیز مورد استفاده قرار می گیرند volatile از کلمه کلیدی

متغیرهای مربوط به اندازه گیری

```
volatile uint16_t previous_rising = 0;
volatile uint16_t Difference_Pulse = 0;
volatile uint16_t Difference_Period = 0;
volatile uint8_t Data_Ready = 0;
```

وظیفه هر متغیر

previous_rising

نگهداری زمان آخرین لبه بالارونده

Difference_Period

نگهداری تعداد تیک های یک دوره کامل

Difference_Pulse

نگهداری تعداد تیک های عرض پالس

Data_Ready

اعلام آماده بودن داده جدید برای پردازش

تابع وقفه Input Capture

```
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM1 &&
        htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2)
    {
        uint16_t current_rising =
            HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);

        uint16_t current_falling =
            HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_2);
```

```

if (previous_rising != 0)
{
    Difference_Period =
current_rising - previous_rising;

    Difference_Pulse =
current_falling - current_rising;

    Data_Ready = 1;
}

previous_rising = current_rising;
}
}

```

اجرا می شود Capture این تابع هنگام وقوع وقفه

ابتدا مقدار ثبت شده لبه بالارونده و لبه پایین رونده خوانده می شود

سپس فاصله زمانی بین دو لبه بالارونده متوالی محاسبه می شود که همان دوره سیگنال است

اختلاف بین لبه بالارونده و پایین رونده نیز عرض پالس را مشخص می کند

برنامه اصلی متوجه می شود که اطلاعات جدید برای پردازش آماده است Data_Ready در انتها با فعال شدن

تعریف پایه های LCD

```

Lcd_PortType ports[] =
{
    GPIOB,
    GPIOB,
    GPIOB,
    GPIOB
};

Lcd_PinType pins[] =
{
    GPIO_PIN_0,
    GPIO_PIN_1,
    GPIO_PIN_2,
    GPIO_PIN_3
};

```

معرفی شده اند LCD در این قسمت پایه های داده

از مد چهار بیتی استفاده شده است تا تعداد پایه های مورد نیاز کاهش پیدا کند

راه اندازی LCD و تایمرها

```
Lcd_HandleTypeDef lcd;
```

```
lcd = Lcd_create  
(  
    ports,  
    pins,  
    GPIOB,  
    GPIO_PIN_4,  
    GPIOB,  
    GPIO_PIN_5,  
    LCD_4_BIT_MODE  
);
```

```
Lcd_clear(&lcd);
```

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
```

```
HAL_TIM_IC_Start(&htim1, TIM_CHANNEL_1);
```

```
HAL_TIM_IC_Start_IT(&htim1, TIM_CHANNEL_2);
```

ساخته و پاک می شود LCD ابتدا

توسط تایمر 2 آغاز می گردد PWM سپس تولید

تایمر 1 فعال می شود تا سیگنال دریافتی اندازه گیری شود Input Capture در ادامه واحد

پردازش اطلاعات و نمایش روی LCD

```
if (Data_Ready == 1)  
{  
    Measured_Freq =  
    72000000.0 / Difference_Period;  
  
    Measured_Duty =  
    ((float)Difference_Pulse /  
    (float)Difference_Period)/2 * 100.0;
```

```

        sprintf
        (
            lcd_buffer,
            "F: %.1f kHz",
            (Measured_Freq / 1000.0)
        );

        Lcd_clear(&lcd);

        Lcd_cursor(&lcd, 0, 0);

        Lcd_string(&lcd, lcd_buffer);

        sprintf
        (
            lcd_buffer,
            "D: %.1f %%",
            Measured_Duty
        );

        Lcd_cursor(&lcd, 1, 0);

        Lcd_string(&lcd, lcd_buffer);

        Data_Ready = 0;
    }

```

در این بخش اطلاعات خام دریافت شده از تایمر به فرکانس و دیوتی سیکل تبدیل می شوند

ابتدا فرکانس محاسبه می شود

سپس دیوتی سیکل از نسبت عرض پالس به دوره تناوب به دست می آید

نمایش داده می شوند LCD در نهایت مقادیر محاسبه شده روی

4. شبیه سازی در Proteus

استفاده شد STM32F103 برای انجام شبیه سازی از میکروکنترلر

روی میکروکنترلر بارگذاری گردید Keil فایل هگز تولید شده توسط

اتصالات مدار

دریافت شد PA15 از پایه PWM خروجی

متصل گردید PA8 از طریق مقاومت 150 اهم به پایه PA15 پایه

تایمر 1 استفاده شد Capture به عنوان ورودی PA8 پایه

متصل شد PB5 تا PBO نیز به پایه های LCD

نتایج شبیه سازی

با مشخصات زیر روی خروجی مشاهده شد PWM پس از اجرای برنامه، سیگنال

$$F = 50kHz$$

$$Duty = 5\%$$

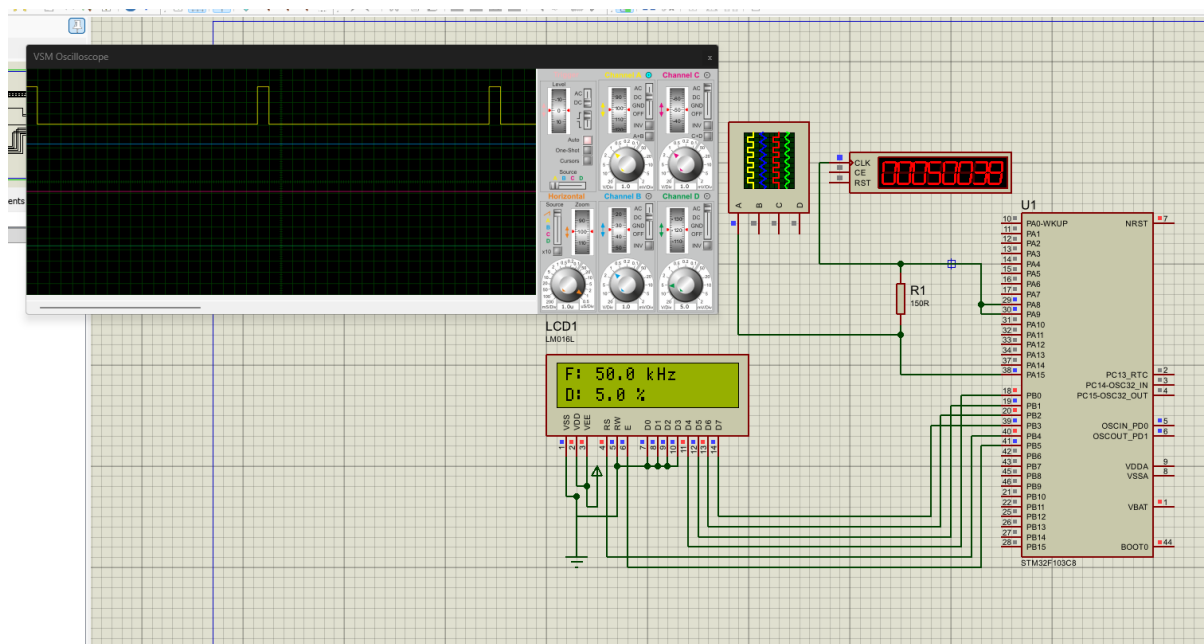
نیز مقادیر اندازه گیری شده را نمایش داد LCD

سطر اول شامل فرکانس اندازه گیری شده

سطر دوم شامل دیوتی سیکل اندازه گیری شده

با فرکانس 50 کیلوهرتز و دیوتی سیکل 5 درصد تولید شد. PWM در این آزمایش با استفاده از تایمر 2 یک سیگنال LCD فرکانس و دیوتی سیکل اندازه گیری گردید و نتایج روی Input Capture سپس توسط تایمر 1 در حالت نمایش داده شد

قابلیت تولید و اندازه STM32 محاسبات تئوری و نتایج عملی با یکدیگر مطابقت داشتند و نشان دادند که تایمرهای با فرکانس بالا و عرض پالس کم را دارند. همچنین مشاهده شد که با افزایش فرکانس PWM گیری دقیق سیگنال های و کاهش دیوتی سیکل، دقت اندازه گیری به تعداد تیک های موجود در هر دوره وابسته شده و محدودیت های سخت افزاری اهمیت بیشتری پیدا می کنند



ویدیو ی شبیه سازی در داخل پوشه سوال موجود می باشد.

8. DMA را فعال کرده و سوال شماره ۶ را به این روش حل کنید. مهم: در خصوص افزایش فرکانس پالس و کاهش عرض آن، و امکان اندازه گیری آنها با فعال بودن DMA، بحث و تشریح کنید.

بخش اول: اطلاعات کلی و محاسبات ریاضی

هدف از این برنامه، تولید یک سیگنال مربعی یا پی دلیوام PWM با فرکانس 5000 Hz و چرخه کار 20٪ توسط تایمر ۲ است. سپس این سیگنال توسط تایمر ۱ با یک روش ترکیبی خوانده و اندازه گیری می شود. در این روش ترکیبی، برای اندازه گیری دوره تناوب از واحد دسترسی مستقیم به حافظه DMA و برای اندازه گیری پهنای پالس از روش سرکشی Polling استفاده شده است.

محاسبات تایمر ۲ برای تولید سیگنال:

فرکانس کلاک سیستم طبق تنظیمات ۷۲ مگاهرتز است. مقسم کلاک یا Prescaler برابر ۷۱ در نظر گرفته شده است تا کلاک ورودی تایمر به ۱ مگاهرتز کاهش یابد.

$$\text{Timer Clock} = \frac{72000000}{71 + 1} = 1000000 \text{ Hz}$$

برای رسیدن به فرکانس 5000 Hz، مقدار رجیستر دوره تناوب یا ARR باید بر اساس فرمول زیر ۱۹۹ تنظیم شود:

$$\text{Frequency} = \frac{1000000}{199 + 1} = 5000 \text{ Hz}$$

برای داشتن چرخه کار 20٪، مقدار رجیستر مقایسه یا Pulse باید بخش مشخصی از کل دوره تناوب باشد:

$$\text{Pulse} = \frac{20 \times 200}{100} = 40$$

محاسبات تایمر ۱ برای خواندن سیگنال:

تایمر ۱ نیز با مقسم کلاک ۷۱ تنظیم شده است، بنابراین هر تیک این تایمر دقیقاً ۱ میکروثانیه طول می کشد. با ثبت زمان بین دو لبه بالارونده متوالی، زمان کل دوره تناوب بر حسب میکروثانیه به دست می آید و با ثبت زمان بین لبه بالارونده اول و لبه پایین رونده، زمان روشن بودن سیگنال محاسبه می شود.

بخش دوم: تنظیمات نرم افزار کیوب ام اکس STM32CubeMX

برای اجرای صحیح این کد، باید تنظیمات زیر در محیط کیوب ام اکس پیکربندی شوند:

• تنظیمات سیستم و کلاک:

- منبع کلاک خارجی HSE روی حالت کریستال فعال تنظیم می شود.
- ضرایب حلقه قفل فاز PLL به گونه ای تنظیم می شوند که کلاک اصلی سیستم روی ۷۲ مگاهرتز قرار گیرد.

- پیکربندی تایمر ۲ تولیدکننده سیگنال:

- منبع کلاک داخلی انتخاب می‌شود.
- کانال ۱ روی حالت PWM Generation CH1 تنظیم می‌گردد.
- مقدار Prescaler برابر ۷۱ و مقدار Counter Period برابر ۱۹۹ تنظیم می‌شود.
- مقدار Pulse برابر ۴۰ قرار می‌گیرد.

- پیکربندی تایمر ۱ اندازه‌گیرنده سیگنال:

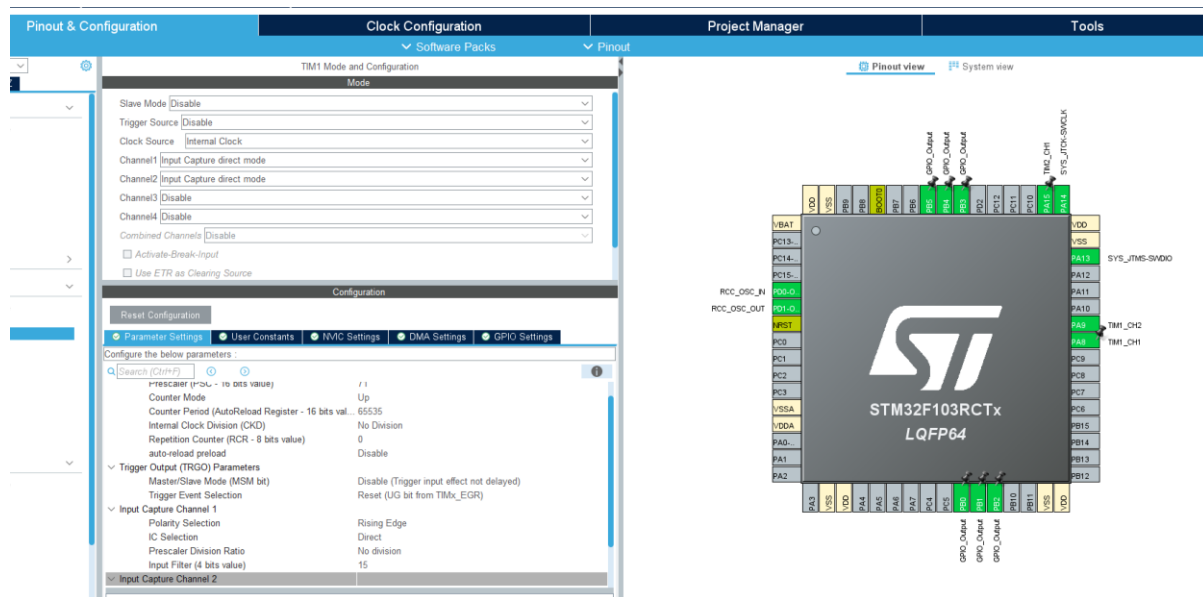
- منبع کلاک داخلی انتخاب می‌شود.
- کانال ۱ روی حالت Input Capture Direct Mode تنظیم شده و حساسیت آن روی لبه بالارونده — Rising Edge — قرار می‌گیرد.
- کانال ۲ روی حالت Input Capture Indirect Mode تنظیم شده و حساسیت آن روی لبه پایین‌رونده — Falling Edge — قرار می‌گیرد.
- مقدار Prescaler برابر ۷۱ و Counter Period برابر ۶۵۵۳۵ تنظیم می‌شود.

- پیکربندی دی‌ام‌ای DMA:

- در تب دی‌ام‌ای مربوط به تایمر ۱، درخواست TIM1_CH1 اضافه می‌شود.
- حالت انتقال روی Normal و اندازه داده‌ها روی اندازه کلمه نیمه — Half Word — تنظیم می‌گردد.

- پیکربندی پایه‌ها GPIO:

- پایه‌های ۰ تا ۵ از پورت بی — GPIOB — به عنوان خروجی برای اتصال به نمایشگر ال‌سی‌دی تنظیم می‌شوند.



بخش سوم: کد keil

در این بخش کدهای نوشته شده در فایل اصلی برنامه را مرحله به مرحله بررسی می‌کنیم:

متغیرهای سراسری:

```
uint16_t rising_buf[2] = {0}; // ذخیره دو لبه بالا رونده از طریق دی‌ام‌ای
```

```
char buffer[30];
```

```
float frequency = 0.0f;
```

```
float duty = 0.0f;
```

آرایه rising_buf برای ذخیره کردن زمان دو لبه بالا رونده متوالی ایجاد شده است. متغیرهای بعدی برای تبدیل اعداد و ذخیره مقادیر نهایی فرکانس و چرخه کار استفاده می‌شوند.

راه اندازی اولیه ماژول‌ها:

```
Lcd_PortType ports[] = { GPIOB, GPIOB, GPIOB, GPIOB };
```

```
Lcd_PinType pins[] = { GPIO_PIN_0, GPIO_PIN_1, GPIO_PIN_2, GPIO_PIN_3 };
```

```
lcd = Lcd_create(ports, pins, GPIOB, GPIO_PIN_4, GPIOB, GPIO_PIN_5, LCD_4_BIT_MODE);
```

```
Lcd_clear(&lcd);
```

این بلوک نمایشگر متنی را در حالت ۴ بیتی راه اندازی کرده و پایه‌های دیتا و کنترل را به پورت بی اختصاص می‌دهد.

شروع به کار تایمرها:

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
```

```
HAL_TIM_Base_Start(&htim1);
```

```
HAL_TIM_IC_Start(&htim1, TIM_CHANNEL_2);
```

ابتدا تولید موج در تایمر ۲ آغاز می‌شود. سپس تایمر ۱ روشن شده و قابلیت تسخیر ورودی برای کانال ۲ در حالت سرکشی فعال می‌گردد.

ثبت دوره تناوب با دی‌ام‌ای:

```
__HAL_TIM_CLEAR_FLAG(&htim1, TIM_FLAG_CC1);
```

```
HAL_TIM_IC_Start_DMA(&htim1, TIM_CHANNEL_1, (uint32_t*)rising_buf, 2);
```

پرچم‌های قبلی پاک شده و دی‌ام‌ای روشن می‌شود تا دقیقاً زمان وقوع دو لبه بالارونده متوالی را در آرایه ثبت کند. یک حلقه شرطی نیز برای جلوگیری از قفل شدن برنامه در نظر گرفته شده است تا اگر سیگنالی وجود نداشت، پس از ۵۰ میلی‌ثانیه خطا صادر شود.

محاسبه اختلاف زمان و کنترل سرریز:

```
if (r1 > r0) period_ticks = r1 - r0;
```

```
else period_ticks = (r1 + 65536) - r0;
```

اگر تایمر در حین شمارش به سقف خود برسد و صفر شود، با اضافه کردن عدد ۶۵۵۳۶ به زمان دوم، خطای محاسباتی اصلاح می‌شود.

یافتن لبه پایین‌رونده:

```
if (__HAL_TIM_GET_FLAG(&htim1, TIM_FLAG_CC2)) {  
    uint32_t ccr2 = HAL_TIM_ReadCapturedValue(&htim1, TIM_CHANNEL_2);  
    if (ccr2 > r0 && ccr2 < r1) {  
        pulse_ticks = ccr2 - r0;  
        fall_ok = 1;  
    }  
    // ...  
}
```

برنامه در یک حلقه کوتاه منتظر پرچم کانال ۲ می‌ماند. به محض وقوع لبه پایین‌رونده، زمان آن خوانده شده و بررسی می‌شود که آیا این لبه در بین دو لبه بالارونده ثبت شده قرار دارد یا خیر. در صورت صحت، پهنای پالس محاسبه می‌گردد.

نمایش اطلاعات:

```
frequency = 1000000.0f / (float)period_ticks;
```

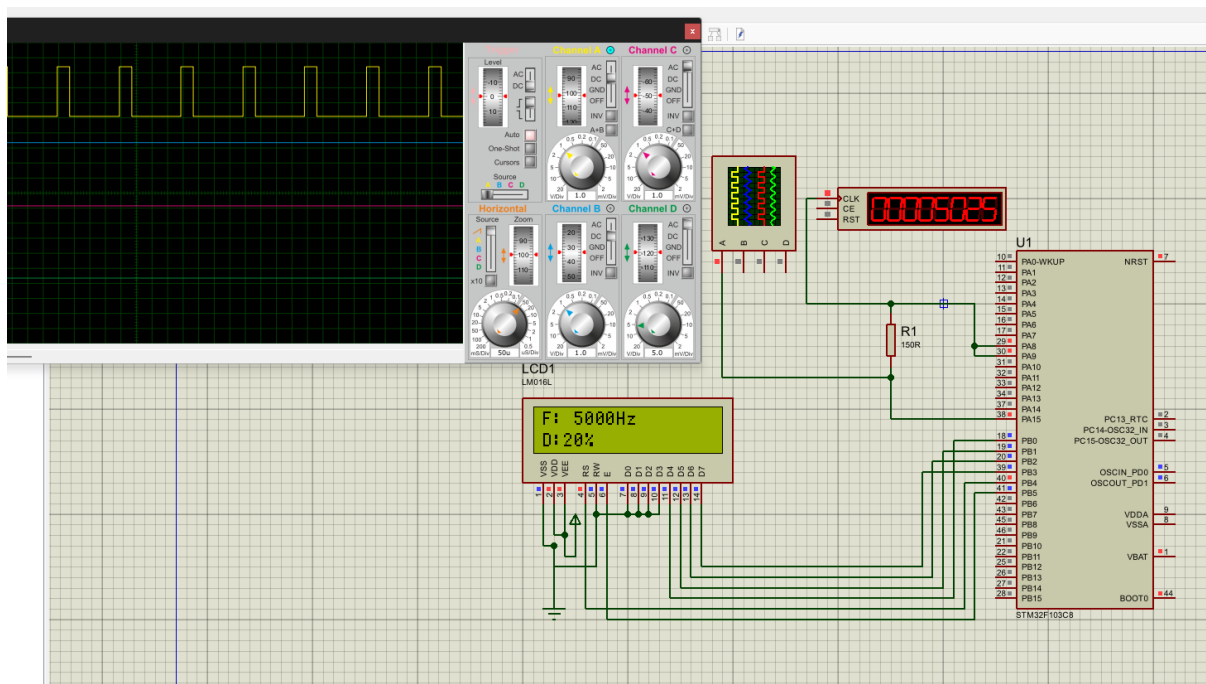
```
duty = ((float)pulse_ticks * 100.0f) / (float)period_ticks;
```

در انتها با استفاده از مقادیر به دست آمده، فرکانس و درصد روشن بودن موج محاسبه شده و توسط دستور `sprintf` برای نمایش روی ال‌سی‌دی آماده می‌شوند.

بخش چهارم: شبیه‌سازی در پروتئوس

برای تست و بررسی این کد در نرم‌افزار شبیه‌ساز پروتئوس، باید مراحل سخت‌افزاری زیر به دقت پیاده‌سازی شوند:

- **میکروکنترلر:** قطعه STM32F103C8T6 یا قطعات مشابه از همین خانواده را در صفحه قرار دهید. در تنظیمات این قطعه، فرکانس پردازنده باید روی ۷۲ مگاهرتز تنظیم شود تا با تنظیمات نرم‌افزاری شما مطابقت داشته باشد.
- **اتصال نمایشگر:** یک ال‌سی‌دی متنی ۱۶ در ۲ را وارد صفحه کنید. پایه‌های D4 تا D7 نمایشگر را به ترتیب به پایه‌های PBO تا PB3 میکروکنترلر متصل نمایید. پایه‌های RS و EN نمایشگر نیز باید به ترتیب به PB4 و PB5 متصل شوند. پایه RW نمایشگر را به زمین GND متصل کنید.
- **اتصال سیگنال‌ها:** پایه خروجی تایمر ۲ را پیدا کنید. از آنجایی که می‌خواهید این سیگنال را اندازه بگیرید، باید این پایه را مستقیماً با یک سیم به پایه ورودی تایمر ۱ که معمولاً PA8 است متصل کنید.
- **ابزار مانیتورینگ:** یک ابزار اسیلوسکوپ از منوی سمت چپ انتخاب کرده و کانال اول آن را به همان سیم ارتباطی بین PA0 و PA8 متصل کنید. هنگام اجرای شبیه‌سازی، می‌توانید روی اسیلوسکوپ مشاهده کنید که هر تناوب ۲۰۰ میکروثانیه طول می‌کشد معادل 5000 Hz و موج در ۴۰ میکروثانیه ابتدایی هر تناوب در وضعیت بالا یا روشن قرار دارد که تایید کننده چرخه کار 20٪ است. همزمان مقادیر دقیق محاسبه شده نیز روی ال‌سی‌دی نمایش داده خواهند شد.



ویدیوی شبیه‌سازی در پوشه سوال موجود می‌باشد.